Глава 11
ПРОГРАММЫ С ГРАФИЧЕСКИМ ИНТЕРФЕЙСОМ

Видел чудеса техники, но такого...

Из к/ф «Иван Васильевич меняет профессию»

В этой главе мы познакомимся с принципами создания *графического интерфейса*. Существуют различные библиотеки, позволяющие создавать приложения с графическим интерфейсом. Мы рассмотрим встроенную библиотеку Tkinter. Для использования ее утилит в программе подключается модуль `tkinter`. Способы использования этой библиотеки и приемы, применяемые при создании графического интерфейса, будем рассматривать на конкретных примерах. Начнем мы с самых простых ситуаций.

Создание простого окна

Это дело очень интересное. И простое!

Из к/ф «Приключения Шерлока Холмса и доктора Ватсона»

Рассмотрим достаточно простую программу, в которой создается пустое окно (оно не содержит ничего, кроме заголовка). Программный код примера представлен в листинге 11.1

Листинг 11.1. Создание простого окна

```
# Импорт класса из пакета:
from tkinter import Tk
# Создание объекта окна:
wnd=Tk()
# Заголовок для окна:
```

```
wnd.title("Простое окно")
# Геометрические размеры окна:
wnd.geometry("250x150")
# Отображение окна на экране:
wnd.mainloop()
```

В результате выполнения этого программного кода на экране появляется окно, представленное на рис. 11.1.

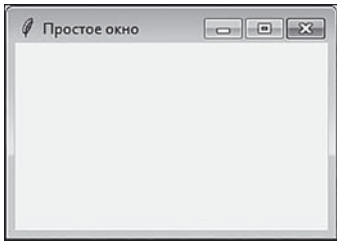


Рис. 11.1. Результат создания простого окна

Ширина окна составляет 250 пикселей, а высота окна равна 150 пикселям. При этом размеры окна можно изменять с помощью курсора мыши.

Проанализируем программный код, с помощью которого создается окно. В первую очередь стоит отметить, что создание окна (как и любого другого графического компонента) означает создание объекта определенного класса. Для окна это класс `Tk`. После того как класс `Tk` импортирован в программу инструкцией `from tkinter import Tk`, командой `wnd=Tk()` создается объект окна, и ссылка на него записывается в переменную `wnd`. Но сам по себе факт создания окна еще не означает, что окно отображается на экране. Для отображения окна на экране из объекта окна вызывается метод `mainloop()` (команда `wnd.mainloop()` в конце программы). Предварительно для окна выполняются минимальные настройки. Так, командой `wnd.title("Простое окно")` задается заголовок для окна. Геометрические размеры окна определяются командой `wnd.geometry("250x150")`.

ПОДРОБНОСТИ

Размеры окна определяются текстовой строкой, в которой указана ширина и высота окна в пикселях. В качестве разделителя в текстовой строке используется буква `"x"`.

При вызове метода `mainloop()` запускается главный цикл обработки событий: программа фактически отображает окно и находится в режиме ожидания действий, выполняемых пользователем. В этом смысле приложения с графическим интерфейсом принципиально отличаются от консольных приложений.

Также стоит отметить, что файлы с реализацией приложений с графическим интерфейсом, сохраняется с расширением `.pyw`.

Окно с меткой и кнопкой

> Мы вам ничего не позволим показывать. Мы вам сами все покажем.
>
> *Из к/ф «Гараж»*

На следующем этапе мы решим более сложную задачу — создадим окно, в котором будут текстовая *метка* и *кнопка*.



ПОДРОБНОСТИ

Хочется верить что любой, кто хоть раз имел дело с программами с графическим интерфейсом, представляет, что такое кнопка. Что касается метки, то это графический компонент, который содержит текст. Это его основное назначение.

Как отмечалось выше, создание любого графического компонента, в том числе кнопки и метки, означает, что на основе определенного класса создается объект. Объект метки создается на основе класса `Label`, а объект кнопки создается на основе класса `Button`. Как эти объекты используются на практике, иллюстрирует программа в листинге 11.2.



Листинг 11.2. Окно с меткой и кнопкой

```
# Импорт классов из пакета:
from tkinter import *
# Создание объекта окна:
wnd=Tk()
# Заголовок для окна:
wnd.title("Окно с кнопкой")
```

```
# Геометрические размеры окна:
wnd.geometry("250x150")
# Окно постоянных размеров:
wnd.resizable(False, False)
# Создание объекта метки:
lbl=Label(wnd, text="Всем привет!", font=("Arial Bold",20))
# Размещение метки в окне:
lbl.place(x=40, y=30)
# Объект копки:
btn=Button(wnd, text="Закрыть", font=("Courier New Bold",13), command=wnd.destroy)
# Размещение объекта кнопки в окне:
btn.place(x=40, y=100, width=170, height=30)
# Отображение окна на экране:
wnd.mainloop()
```

При выполнении этой программы на экране появляется окно, показанное на рис. 11.2.

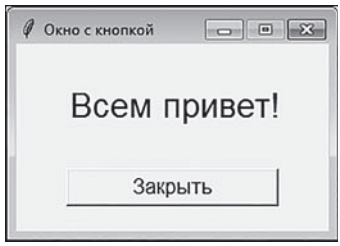


Рис. 11.2. Окно с меткой и кнопкой

В центральной части окна отображается текст, а в нижней части окна расположена кнопка **Закрыть**, нажатие которой приводит к закрытию окна и завершению выполнения программы.

НА ЗАМЕТКУ

Создается окно постоянных размеров — размеры этого окна нельзя изменить с помощью курсора мыши.

Помимо уже знакомых нам команд, программа содержит и новые инструкции. Так, мы хотим на этот раз создать окно постоянных размеров, поэтому после создания объекта окна `wnd`, командой `wnd.resizable(False, False)` для окна устанавливается режим постоянных размеров.



ПОДРОБНОСТИ

Два аргумента `False`, переданные методу `resizable()`, означают запрет изменять размеры окна соответственно по горизонтали и вертикали.

Объект кнопки `lbl` создается командой `lbl=Label(wnd, text="Всем привет!", font=("Arial Bold",20))`. Первый аргумент `wnd` конструктора класса `Label` означает, что метка будет размещаться в окне `wnd`. Аргумент `text` определяет надпись, содержащуюся в текстовой метке. Наконец, аргумент `font` задает шрифт, которым будет отображаться текст в метке. Значением этого аргумента указан кортеж с элементами `"Arial Bold"` и `20`, определяющими соответственно тип и размер шрифта. Размещается метка в окне с помощью команды `lbl.place(x=40, y=30)`. Аргументы метода `place()` определяют координаты метки в области окна. Координаты задаются для левого верхнего угла метки по отношению к левому верхнему углу окна. Координаты задаются в пикселях. Первая горизонтальная координата отсчитывается вдоль горизонтали слева направо, а вторая вертикальная координата отсчитывается сверху вниз.



НА ЗАМЕТКУ

Этот же способ определения координат применяется и для прочих графических компонентов.



ПОДРОБНОСТИ

Существует несколько способов, которыми компонент может добавляться в контейнер (в данном случае контейнером является окно). Если для размещения компонента используется метод `place()`, то координаты компонента указываются явно. Можно также использовать методы `pack()` и `grid()`. С помощью метода `pack()` компонент размещается путем определения положения компонента по отношению к сторонам контейнера. Если используется метод

grid(), то положение компонента определяется указанием ячейки в воображаемой таблице, которая размещена в области окна.

Объект кнопки btn создается командой btn=Button(wnd, text="Закрыть", font=("Courier New Bold",13), command=wnd.destroy). Первый аргумент конструктора класса Button определяет принадлежность кнопки. Аргумент text задает название кнопки. Аргумент font определяет шрифт, применяемый при отображении названия кнопки. Важна роль аргумента command. Он определяет действие, выполняемое при нажатии кнопки. Значение wnd.destroy аргумента означает, что при нажатии кнопки из объекта wnd будет вызываться метод destroy(). При вызове этого метода окно закрывается (и, как следствие, программа прекращает выполнение). Наконец, размещается кнопка в окне командой btn.place(x=40, y=100, width=170, height=30). Аргументы x и y определяют координаты кнопки в области окна, а аргументы width и height задают соответственно ширину и высоту кнопки (в пикселях).

НА ЗАМЕТКУ

Вызов метода destroy() приводит к удалению объекта окна, но это не означает автоматического завершения программы. Если нужно завершить всю программу, используют метод quit() (или иные решения, в зависимости от контекста задачи). Если нужно окно убрать с экрана (не уничтожая объект окна), то используют метод withdraw(). Чтобы снова отобразить скрытое окно, используют метод deiconify().

Использование текстового поля

Фигуры, может, и нет, а характер — налицо.

Из к/ф «Девчата»

Текстовое *поле* — графический компонент, который предназначен для ввода текста. Текстовые поля реализуются в виде объектов класса Entry. Далее рассматривается программа, в которой отображается окно с текстовым полем. В это поле пользователю предлагается ввести свое имя. Окно представлено на рис. 11.3.

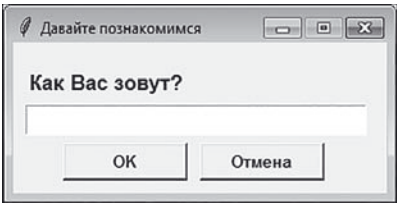


Рис. 11.3. Окно с полем ввода появляется при запуске программы на выполнение

Окно содержит две кнопки: **OK** и **Отмена**. Если пользователь нажимает кнопку **Отмена** или закрывает окно с помощью системной пиктограммы в правом верхнем углу окна, то окно закрывается, и программа на этом прекращает выполнение. Если пользователь вводит в поле текст и нажимает кнопку **OK**, то введенный пользователем текст считывается, запоминается, и затем после первого окна отображается еще одно окно с сообщением, в котором содержится текст, введенный пользователем в поле ввода в первом окне. На рис. 11.4 показано окно с именем, которое ввел пользователь, перед нажатием кнопки **OK**.

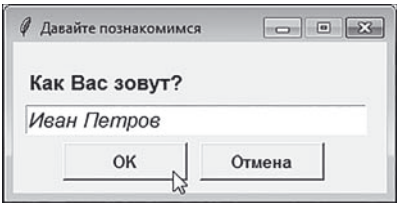


Рис. 11.4. Окно с именем в поле ввода перед нажатием кнопки **OK**

После нажатия кнопки **OK** появится окно, представленное на рис. 11.5.

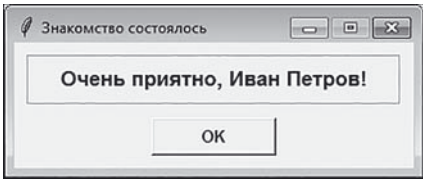


Рис. 11.5. Окно с сообщением появляется после нажатия кнопки **OK** в первом окне

А теперь рассмотрим программный код в листинге 11.3.

Листинг 11.3. Использование поля ввода

```
from tkinter import *
# Функция для обработки нажатия кнопки:
def clicked():
    global t
    # Считывается содержимое текстового поля:
    t=txt.get()
    # Закрывается окно:
    wnd.destroy()
# Создается объект для первого окна:
wnd=Tk()
# Параметры окна:
wnd.title("Давайте познакомимся")
wnd.geometry("300x120")
wnd.resizable(False, False)
# Шрифты:
fnt_1=("Arial",13,"bold")
fnt_2=("Arial",13,"italic")
fnt_3=("Arial",10,"bold")
# Переменная для записи текста из поля ввода:
t=""
# Создание объекта для текстовой метки:
lbl=Label(master=wnd, text="Как Вас зовут?")
# Шрифт для метки:
lbl.configure(font=fnt_1)
# Добавление метки в окно:
lbl.place(x=10, y=20)
# Создание объекта для поля ввода:
txt=Entry(master=wnd, width=30)
# Шрифт для поля ввода:
txt.configure(font=fnt_2)
# Размещение текстового поля в окне:
```

```
txt.place(x=10, y=50)
# Создание объектов для кнопок:
btn_1=Button(master=wnd, text="OK")
btn_2=Button(master=wnd, text="Отмена")
# Параметры кнопок:
btn_1.configure(font=fnt_3)
btn_1.configure(command=clicked)
btn_2.configure(font=fnt_3)
btn_2.configure(command=wnd.destroy)
# Размещение кнопок в окне:
btn_1.place(x=40, y=80, width=100, height=30)
btn_2.place(x=150, y=80, width=100, height=30)
# Отображение первого окна на экране:
wnd.mainloop()
# Если пользователь ввел текст:
if t!="":
    # Создание объекта для второго окна:
    msg=Tk()
    # Параметры второго окна:
    msg.title("Знакомство состоялось")
    msg.geometry("320x100")
    msg.resizable(False, False)
    # Метка с сообщением для второго окна:
    lbl=Label(master=msg, text="Очень приятно, "+t+"!", relief=GROOVE)
    # Шрифт для метки:
    lbl.configure(font=fnt_1)
    # Размещение метки во втором окне:
    lbl.place(x=10, y=10, height=40, width=300)
    # Создание объекта кнопки:
    btn=Button(master=msg, text="OK")
    # Шрифт для кнопки:
    btn.configure(font=fnt_3)
```

```
# Метод для обработки нажатия кнопки:
btn.configure(command=msg.destroy)
# Размещение кнопки во втором окне:
btn.place(x=110, y=60, width=100, height=30)
# Отображение второго окна на экране:
msg.mainloop()
```

Первое окно (объект `wnd`) создается уже знакомым нам способом. В окно добавляется текстовая метка, поле ввода и две кнопки. Для удобства мы создаем три кортежа (`fnt_1`, `fnt_2` и `fnt_3`), определяющие используемые в программе шрифты. Каждый кортеж содержит три элемента: название шрифта, размер шрифта и стиль шрифта. Также мы определяем переменную `t` с пустой текстовой строкой в качестве начального значения. В эту переменную планируется записывать текст, который пользователь введет в поле ввода.

Объект метки создается и размещается в окне практически так же, как мы это делали раньше.



ПОДРОБНОСТИ

Если до этого мы просто указывали первый аргумент, то теперь он передается по ключу `master`. Шрифт для метки мы задаем отдельно. Поэтому из объекта метки вызывается метод `configure()` и значением аргумента `font` указывается переменная, определяющая шрифт, который и будет применяться для метки. Эти же замечания относятся и к созданию прочих компонентов.

Объект для поля ввода создается командой `txt=Entry(master=wnd, width=30)`. Аргумент `width` в данном случае определяет ширину поля в символах (которые в это поле можно ввести). Шрифт для текстового поля задаем командой `txt.configure(font=fnt_2)`. Этот шрифт будет применяться к тексту, вводимому в поле. Для размещения поля в окне использована команда `txt.place(x=10, y=50)`, в которой аргументы метода `place()`, как и ранее, определяют положение (координаты в пикселях) левого верхнего угла поля по отношению к левому верхнему углу окна.

Кроме поля ввода создаются две кнопки. Обработчики событий для кнопок мы регистрируем вызовом из объекта соответствующей кнопки

метода `configure()`, указав значением аргумента `command` имя функции (или метода), вызываемых при нажатии кнопки. Для одной из кнопок таким значением указана ссылка на метод `destroy()` объекта окна `wnd`. В результате вызова этого метода окно убирается с экрана (но программа при этом автоматически работу не прекращает). Для другой кнопки обработчиком события нажатия на кнопке указана функция `clicked()`. Функция описана в программе следующим образом. Командой `t=txt.get()` в глобальную переменную `t` записывается значение, содержащееся в поле ввода. Для получения этого значения из объекта поля `txt` вызывается метод `get()`. После этого командой `wnd.destroy()` окно закрывается.

Для размещения кнопок в окне используется метод `place()`, в качестве аргументов которому передаются координаты кнопки (`x` и `y`) и ее размеры (ширина `width` и высота `height`), указанные в пикселях.

После того как метка, поле и кнопки добавлены в окно, командой `wnd.mainloop()` оно отображается на экране.

Следующей командой размещен условный оператор. Он будет выполняться только после того, как закроется первое окно (которое отображается предыдущей командой).



ПОДРОБНОСТИ

При вызове метода `mainloop()` из объекта окна фактически запускается бесконечный цикл, в рамках которого интерпретатор ожидает и обрабатывает операции, выполняемые пользователем с окном. Поэтому до выполнения следующей команды дело не доходит. Чтобы начала выполняться следующая команда, необходимо, чтобы окно было закрыто.

В условном операторе проверяется условие `t!=""`. Оно истинно в том случае, если переменная ссылается на непустую текстовую строку. Это происходит в случае, если в первом окне пользователь в поле ввода указал текст и нажал кнопку **OK**. Если так, то командой `msg=Tk()` создается объект для нового кона, выполняется настройка его параметров, а также создаются и размещаются в окне метка и кнопка. Текст для метки вычисляется выражением `"Очень приятно, "+t+"!"`, в котором используется значение переменной `t`. Также мы используем аргумент `relief` со значением `GROOVE`. С помощью этого аргумента задается режим отображения рамки вокруг метки.



ПОДРОБНОСТИ

То, что мы ссылку на объект метки записываем в переменную `lbl`, которую уже использовали ранее, проблемой не является. Первое окно мы больше не используем, а в переменную `lbl` записывается ссылка на новый объект.

Аргумент `relief`, как отмечалось, определяет рамку вокруг метки (точнее, способ выделения метки на фоне контейнера). Возможные значения: `FLAT` (фактически рамка отсутствует), `RAISED` (эффект «приподнятой» метки), `SUNKEN` (эффект «вдавленной» метки), `GROOVE` (вдавленная рамка вокруг метки) и `RIDGE` (приподнятая рамка вокруг метки).

Также стоит еще раз заметить, что аргументы `width` и `height` (равно как аргументы `x` и `y`) в методе `place()` задаются в пикселях.

Кнопка для окна определяется так, что при ее нажатии из объекта окна `msg` вызывается метод `destroy()`, в результате чего окно закрывается и, поскольку на этом код исчерпан, заканчивается выполнение программы. Для отображения окна на экране использована команда `msg.mainloop()`.



НА ЗАМЕТКУ

Если нужно работать с текстами больших объемов, можно использовать текстовую область, которая реализуется как объект класса `Text`.

Раскрывающийся список

Огласите весь список, пожалуйста.

Из к/ф «Операция Ы и другие приключения Шурика»

Еще один полезный графический компонент, который позволяет создавать эффективные приложения с графическим интерфейсом, — *раскрывающийся список*. Для реализации раскрывающегося списка мы воспользуемся классом `Combobox` из подпакета `ttk` пакета `tkinter`.



ПОДРОБНОСТИ

Для использования этого класса в программе, рассматриваемой далее, есть инструкция `from tkinter.ttk import Combobox`.

Что касается самой программы, то она такая: при запуске программы на выполнение появляется окно, показанное на рис. 11.6.

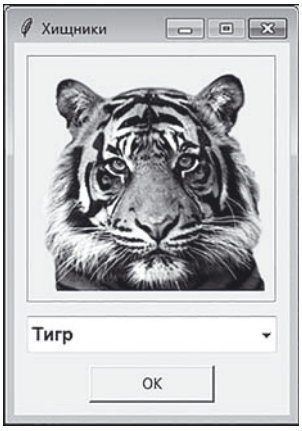


Рис. 11.6. Окно с изображением тигра

В центральной части окна содержится изображение тигра, а внизу под изображением есть раскрывающийся список (сначала в нем выбрано значение **Тигр**) и кнопка **ОК**. При нажатии кнопки окно закрывается. А если нажать пиктограмму со стрелочкой в раскрывающемся списке, то выпадет его содержимое: названия трех животных (**Тигр**, **Лев** и **Медведь**). Окно с раскрытым списком представлено на рис. 11.7.



Рис. 11.7. Выбор нового значения в раскрывающемся списке

После выбора пункта в раскрывающемся списке в центральной области окна автоматически отображается соответствующая картинка.

На рис. 11.8 показано окно с изображением льва (в раскрывающемся списке выбран пункт **Лев**).



Рис. 11.8. Окно с изображением льва

Как будет выглядеть окно, если в списке выбрать пункт **Медведь**, показано на рис. 11.9.

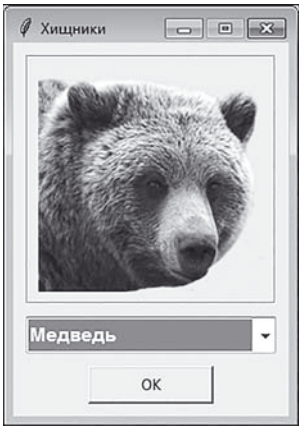


Рис. 11.9. Окно с изображением медведя

Прежде чем приступить к рассмотрению программного кода, сразу отметим несколько моментов, а именно: для размещения изображения в области окна мы используем метку. Хотя она формально называется текстовой, но в действительности может содержать не только текст, но и изображения. Ссылка на объект изображения присваивается

в качестве значения аргументу `image` в конструкторе класса `Label`. Объект изображения будет создаваться на основе класса `PhotoImage`, а в качестве аргументов конструктору передаются ссылка на объект окна (значение аргумента `master`) и текст с полным путем к файлу изображения (значение аргумента `file`).

НА ЗАМЕТКУ

Для корректной работы программы в каталоге `D:\Books\Python\Pictures\` должны находиться файлы `tiger.png`, `lion.png` и `bear.png` с изображениями животных. Каждое изображение имеет размер 180 пикселей в ширину и 180 — в высоту и прозрачный фон (хотя это и не принципиально). Если читатель планирует использовать иные файлы (но с тем же расширением), размещенные в другом месте, то в программу нужно будет внести соответствующие изменения.

Теперь рассмотрим программный код, представленный в листинге 11.4.

Листинг 11.4. Раскрывающийся список

```
# Импорт содержимого пакета:
from tkinter import *
# Импорт класса из подпакета:
from tkinter.ttk import Combobox
# Функция для обработки события, связанного
# с выбором пункта в раскрывающемся списке:
def change(evt):
    # Считывание значения, выбранного в списке:
    t=cb.get()
    # Определение изображения по выбранному значению:
    for k in range(len(names)):
        # Если выбранное значение совпадает
        # с текстом в списке:
        if t==names[k]:
            # Новое изображение в метке:
            lbl.configure(image=imgs[k])
            # Завершение оператора цикла:
```

```
            break
# Переменная с путем к файлам изображений:
path="D:\\Books\\Python\\Pictures\\"
# Названия животных:
names=["Тигр","Лев","Медведь"]
# Названия файлов с изображениями:
files=["tiger.png","lion.png","bear.png"]
# Создание объекта окна:
wnd=Tk()
# Параметры окна:
wnd.title("Хищники")
wnd.geometry("220x300")
wnd.resizable(False, False)
# Список с объектами для изображений:
imgs=[PhotoImage(file=path+f) for f in files]
# Индекс для выбранного в начале пункта:
index=0
# Создание объекта метки:
lbl=Label(wnd, image=imgs[index])
# Параметры метки и ее размещение в окне:
lbl.configure(relief=GROOVE)
lbl.place(x=10, y=10, width=200, height=200)
# Создание объекта для раскрывающегося списка:
cb=Combobox(wnd, state="readonly")
# Содержимое раскрывающегося списка:
cb.configure(values=names)
# Выбранное в начале значение (пункт):
cb.current(index)
# Шрифт для раскрывающегося списка:
cb.configure(font=("Arial",11,"bold"))
# Определение обработчика события, связанного
# с выбором в списке нового значения:
```

```
cb.bind("<<ComboboxSelected>>", change)
# Размещение списка в окне:
cb.place(x=10, y=220, width=200, height=30)
# Создание объекта кнопки:
btn=Button(wnd, text="OK")
# Определение обработчика для события,
# связанного с нажатием кнопки:
btn.configure(command=wnd.destroy)
# Размеры кнопки и ее размещение в окне:
btn.place(x=60, y=260, width=100, height=30)
# Отображение окна на экране:
wnd.mainloop()
```

Кроме инструкций импорта содержимого пакета `tkinter` и импорта класса `Combobox` из подпакета `ttk`, программа содержит описание функции `change()`, несколько глобальных списков и переменных, а также собственно команды, которыми создается окно и все его содержимое. Так, в переменную `path` мы записываем путь к каталогу, в котором хранятся файлы с изображениями. В нашем случае это текст `"D:\\Books\\Python\\Pictures\\"`.

> **НА ЗАМЕТКУ**
>
> Если планируется размещать файлы с изображениями в ином месте, то в качестве значения переменной `path` следует указать текстовое значение с путем к соответствующему каталогу.
>
> Также напомним, что для включения в текст обратной косой черты `\` используется комбинация `\\` из двух обратных косых черт — иначе следующий после обратной косой черты символ будет восприниматься как управляющий.

Список `names` содержит названия животных. Он определяет пункты раскрывающегося списка. Список `files` содержит название файлов с изображениями животных. Между списками `names` и `files` должно быть взаимно однозначное соответствие: индекс элемента с названием животного в списке `names` должен быть таким же, как и индекс элемента с названием файла с изображением животного в списке `files`.

НА ЗАМЕТКУ

Чтобы расширить содержимое раскрывающегося списка, достаточно в список `names` добавить названия животных (новые пункты раскрывающегося списка), а в список `files` добавить названия файлов с изображениями этих животных (и, разумеется, сами файлы должны быть добавлены в каталог с изображениями).

После того как создан объект окна `wnd` и заданы его основные параметры (заголовок, размеры и переход в режим окна постоянных размеров), командой `imgs=[PhotoImage(file=path+f) for f in files]` создается список с объектами изображений, которые будут использоваться для отображения в окне.

ПОДРОБНОСТИ

Для использования изображения в приложении с графическим интерфейсом на основе этого изображения необходимо создать специальный объект. Объект создается на основе класса `PhotoImage`. Значением аргумента `file` указывается путь к файлу, на основе которого создается объект изображения.

Переменная `index` с начальным нулевым значением определяет пункт из раскрывающегося списка и соответствующее ему изображение, которые будут показаны в окне в самом начале.

Метку создаем командой `lbl=Label(wnd, image=imgs[index])`. В данном случае примечательно то, что в качестве значения аргументу `image` передается ссылка на объект изображения из списка `imgs`. Индекс объекта определяется значением переменной `index`.

Командой `lbl.configure(relief=GROOVE)` для метки задается режим отображения вдавленной рамки вокруг метки. Командой `lbl.place(x=10, y=10, width=200, height=200)` задаются ширина и высота метки, а также ее положение в окне.

НА ЗАМЕТКУ

Напомним, что используются изображения размерами 180 пикселей в ширину и 180 пикселей в высоту. Размеры метки немного больше: 200 пикселей в ширину и 200 пикселей в высоту.

Объект для раскрывающегося списка создается командой `cb=Combobox(wnd, state="readonly")`. Значение `"readonly"` для аргумента `state` означает, что поле раскрывающегося списка с выбранным пунктом не будет доступно для редактирования (то есть пункт в раскрывающемся списке можно будет выбрать, но нельзя будет ввести название этого пункта с помощью клавиатуры).

Содержимое раскрывающегося списка определяется командой `cb.configure(values=names)`. Здесь аргументу `values` в качестве значения присваивается ссылка на список `names`, содержащий названия животных. Именно они будут отображаться в раскрывающемся списке. Наконец, командой `cb.current(index)` определяется значение (пункт) в раскрывающемся списке, который будет выбран в самом начале. Этот пункт определяется элементом в списке `names` с таким же индексом, как и у элемента в списке изображений, который указан для отображения в метке. Также мы задаем шрифт для раскрывающегося списка (команда `cb.configure(font=("Arial",11,"bold"))`). Этот шрифт будет применяться для отображения названия выбранного в раскрывающемся списке пункта.

По сравнению с рассмотренными выше графическими компонентами, изменился способ, которым мы задаем для раскрывающегося списка обработчик события, связанного с выбором в раскрывающемся списке нового пункта. На этот раз мы используем метод `bind()`. В качестве аргументов методу передаются название события, для которого регистрируется обработчик, и функция/метод, которые вызываются для обработки этого события. Мы хотим реализовать обработку события `<<ComboboxSelected>>`, которое состоит в том, что в раскрывающемся списке выбран пункт. Для обработки события будет вызываться функция `change()`. Поскольку при обработке события объект этого события передается в функцию, то такая функция должна быть описана с аргументом (аргумент события). Поэтому функцию `change()` мы описываем с аргументом — правда, мы его не используем.



ПОДРОБНОСТИ

Вообще объект события содержит полезную информацию о событии — например координаты курсора мыши в момент наступления события и ряд других характеристик. В принципе эту информацию можно «извлечь» и использовать. Нам она не нужна, но указать аргумент в описании функции мы все равно должны.

Функция `change()`, которая вызывается при выборе нового пункта в раскрывающемся списке, описана следующим образом. Сначала командой `t=cb.get()` считывается и записывается в переменную `t` значение, выбранное в раскрывающемся списке. После этого запускается оператор цикла, в котором выполняется поиск элемента с таким же названием, что записано в переменную `t`. Как только совпадение найдено (истинно условие `t==names[k]` в условном операторе), командой `lbl.configure(image=imgs[k])` для метки задается новое изображение. Это объект из списка `imgs` с таким же индексом `k`, что и индекс элемента из списка `names`, который выбран в раскрывающемся списке. После этого работа оператора цикла завершается с помощью инструкции `break`.

Во всем остальном программный код простой: мы размещаем метку в окне (команда `cb.place(x=10, y=220, width=200, height=30)`), создаем кнопку (команда `btn=Button(wnd, text="OK")`), определяем обработчик для события, связанного с нажатием кнопки (команда `btn.configure(command=wnd.destroy)`), и тоже размещаем ее в окне (команда `btn.place(x=60, y=260, width=100, height=30)`). Наконец, окно отображается на экране (использована команда `wnd.mainloop()`).

Опции, переключатели и другие компоненты

Первый раз таких единоличников вижу.

Из к/ф «Девчата»

В следующем примере мы познакомимся с такими графическими компонентами, как *опции*, *переключатели*, *слайдер* и *статический список*. Кроме этого, для размещения компонентов мы будем использовать *панели*, а также узнаем, как компоненты размещаются в контейнере с помощью метода `pack()`.

Сначала мы рассмотрим результат выполнения программы — после этого нам будет легче проанализировать программный код. Итак, при запуске программы на выполнение на экране появляется окно, представленное на рис. 11.10.

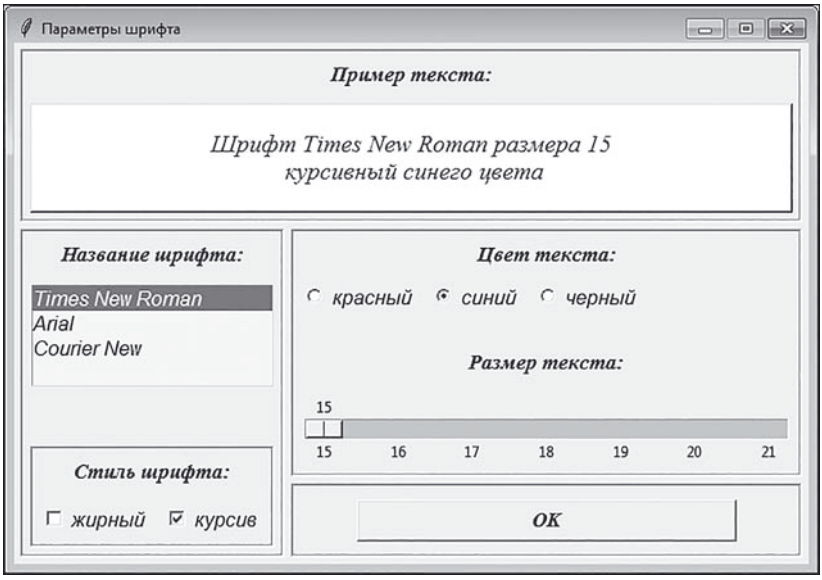


Рис. 11.10. Окно для настроек параметров шрифта

В верхней части окна размещена панель с надписью **Пример текста** и белой областью с текстом. Текст отображается синим цветом курсивным шрифтом **Times New Roman** размера **15**. Собственно сам текст содержит эту информацию. В левой части окна расположена панель с надписью **Название шрифта** и списком для выбора названия шрифта из трех пунктов. В нижней части этой панели, под надписью **Стиль шрифта**, расположены две опции для выбора стиля шрифта (по умолчанию установлен флажок опции **курсив**, и не установлен флажок опции **жирный**).

Справа в области окна под надписью **Цвет текста** расположена панель с группой переключателей (**красный**, **синий** и **черный** — по умолчанию выбран пункт **синий**), предназначенных для выбора цвета шрифта. Также панель под надписью **Размер текста** содержит слайдер для выбора размера шрифта. Далее внизу размещена еще одна панель с кнопкой **ОК**, нажатие которой приводит к закрытию окна.

При изменении настроек в окне автоматически изменения вносятся в шаблонный текст: меняется как содержимое текста, так и шрифт, которым отображается этот текст (в том числе и цвет текста). На рис. 11.11 проиллюстрировано состояние окна, когда выбран шрифт **Arial**, установлены опции **жирный** и **курсив**, а также выбран пункт **красный** в группе переключателей, предназначенных для выбора цвета.

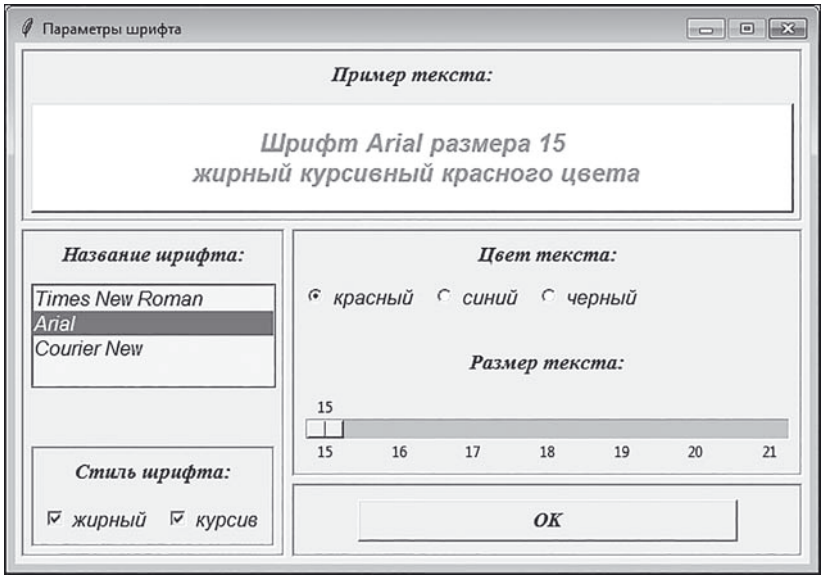


Рис. 11.11. Результат изменения параметров шрифта

Изменение положения слайдера приводит к изменению размера текста. На рис. 11.12 показан ситуация, когда используется шрифт **Courier New**, флажки опций выбора стиля отменены, выбран черный цвет для отображения текста, а слайдер установлен на значении **20**.

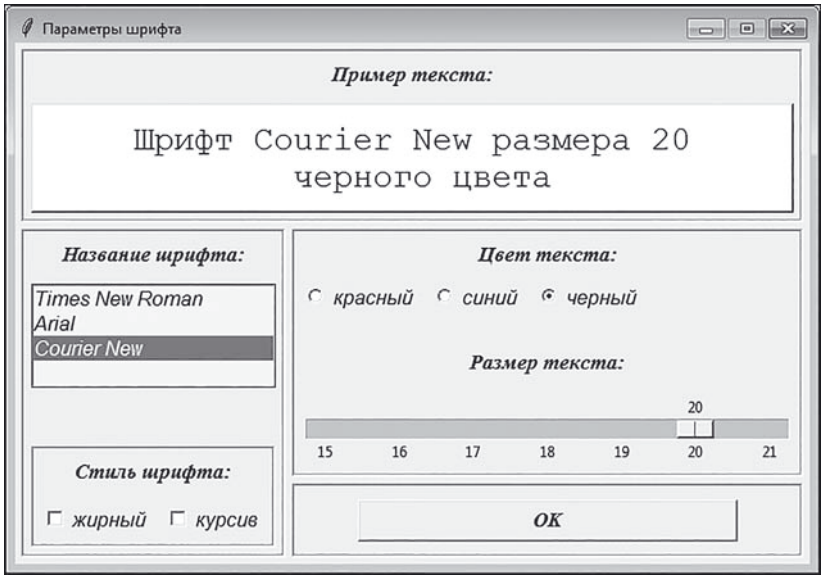


Рис. 11.12. Результат изменения размера, цвета и стиля шрифта

Перед рассмотрением кода сделаем несколько общих замечаний. В первую очередь это относится к классам, на основе которых создаются объекты графических компонентов. Итак, объекты для переключателей создаются на основе класса `Radiobutton`. Класс `Scale` используется для создания объекта слайдера. Объекты для опций создаются на основе класса `Checkbutton`. Класс `Listbox` служит основой для создания объекта статического списка. Кроме этих компонентов, как отмечалось выше, мы используем такой графический компонент, как панель. Объект панели создается на основе класса `Frame`.



ПОДРОБНОСТИ

Главное назначение панели — быть контейнером для размещения других компонентов. Панель представляет собой прямоугольную область, в которую добавляются компоненты, а затем эту панель (вместе с компонентами, которые она содержит) можно добавить в другую панель или окно.

Как иллюстрацию в рассматриваемом далее примере для компоновки (размещения) компонентов на панелях используется метод `pack()` (сами панели, за исключением одной, размещаются в окне с помощью метода `place()`).



ПОДРОБНОСТИ

Если в случае использования метода `place()` положение компонента в контейнере указывается явно, то в случае использования метода `pack()` определяется общее положение компонента в контейнере (например, слева, справа, вверху или внизу в области контейнера).

Также в некоторых случаях мы используем для изменения содержимого элементов управления и считывания состояния элементов управления специальные «переменные» (а если точнее, то объекты) класса `StringVar`. Интересующий нас программный код представлен в листинге 11.5.



Листинг 11.5. Определение параметров шрифта

```
from tkinter import *

# Функция для определения характеристик шрифта:

def getFont():
```

```
    # Пустой список:
    res=[]
    # Название шрифта:
    name=lst.get(lst.curselection())
    # Размер шрифта:
    size=scl.get()
    # Добавление элементов в список:
    res.append(name)
    res.append(size)
    # Если установлена опция применения
    # жирного стиля:
    if bold.get()!="":
        res.append(bold.get())
    # Если установлена опция применения
    # курсивного стиля:
    if italic.get()!="":
        res.append(italic.get())
    # Результат функции:
    return res
# Функция для применения параметров шрифта:
def setAll(*args):
    # Вычисление шрифта:
    fnt=getFont()
    # Применение шрифта к тексту в окне:
    lbl.configure(font=fnt)
    # Определение цвета для текста в окне:
    lbl.configure(fg=color.get())
    # Определение шаблонного текста
    # для отображения в окне:
    txt="\nШрифт "
    # Название шрифта:
    txt+=fnt[0]
```

```
    # Размер шрифта:
    txt+=" размера "+str(fnt[1])+"\n"
    # Если применяется жирный стиль:
    if "bold" in fnt:
        txt+=" жирный"
    # Если применяется курсивный стиль:
    if "italic" in fnt:
        txt+=" курсивный"
    # Цвет шрифта:
    if color.get()=="red":
        txt+=" красного"
    elif color.get()=="blue":
        txt+=" синего"
    else:
        txt+=" черного"
    txt+=" цвета\n"
    # Отображение текста в окне:
    text.set(txt)
# Списки с характеристиками шрифтов:
fnt_1=["Arial",12,"italic"]
fnt_2=["Times New Roman",13,"bold","italic"]
# Список с названиями шрифтов для статического списка:
fonts=["Times New Roman","Arial","Courier New"]
# Минимальный размер шрифта:
min_size=15
# Максимальный размер шрифта:
max_size=21
# Ширина и высота окна:
W=640
H=420
# Высота панели с шаблонным текстом:
Hf=140
```

```
# Ширина и высота панели со статическим списком:
Wl=W/3
Hl=H-Hf-15
# Высота панели с кнопкой:
Hb=60
# Ширина и высота панели со слайдером и переключателями:
Ws=W-Wl-15
Hs=Hl-Hb-5
# Создание окна:
wnd=Tk()
wnd.title("Параметры шрифта")
wnd.geometry(str(W)+"x"+str(H))
wnd.resizable(False, False)
# Создание панелей:
frm_scale=Frame(wnd, bd=3, relief=GROOVE)
frm_text=Frame(wnd, bd=3, relief=GROOVE)
frm_btn=Frame(wnd, bd=3, relief=GROOVE)
frm_list=Frame(wnd, bd=3, relief=GROOVE)
frm_check=Frame(frm_list, bd=3, relief=GROOVE)
# Переменные для определения текстового
# содержимого в элементах управления:
text=StringVar()
color=StringVar()
bold=StringVar()
italic=StringVar()
# Создание текстовых меток:
lbl_text=Label(frm_text, text="Пример текста:", font=fnt_2)
lbl_color=Label(frm_scale, text="Цвет текста:", font=fnt_2)
lbl_size=Label(frm_scale, text="Размер текста:", font=fnt_2)
lbl_font=Label(frm_list, text="Название шрифта:", font=fnt_2)
lbl_style=Label(frm_check, text="Стиль шрифта:", font=fnt_2)
# Метка для отображения шаблонного текста:
```

```
lbl=Label(frm_text, textvariable=text)
# Фон и рамка для метки:
lbl.configure(bg="white", relief=RAISED)
# Переключатели:
rb_1=Radiobutton(frm_scale, text="красный", variable=color)
rb_1.configure(value="red", font=fnt_1)
rb_2=Radiobutton(frm_scale, text="синий", variable=color)
rb_2.configure(value="blue", font=fnt_1)
rb_3=Radiobutton(frm_scale, text="черный", variable=color)
rb_3.configure(value="black", font=fnt_1)
# Устанавливается переключатель:
color.set("blue")
# Создание слайдера:
scl=Scale(frm_scale, orient=HORIZONTAL)
# Диапазон изменения значений:
scl.configure(from_=min_size, to=max_size)
# Интервал для отображения подписей
# и шаг дискретности для положения ползунка:
scl.configure(tickinterval=1, resolution=1)
# Обработчик для события, связанного
# с изменением положения слайдера:
scl.config(command=setAll)
# Создание опций и настройка их параметров:
chb_1=Checkbutton(frm_check, text="жирный", variable=bold)
chb_1.configure(onvalue="bold", offvalue="", font=fnt_1)
chb_2=Checkbutton(frm_check, text="курсив", variable=italic)
chb_2.configure(onvalue="italic", offvalue="", font=fnt_1)
# Начальное состояние опций:
bold.set("")
italic.set("italic")
# Создание статического списка:
lst=Listbox(frm_list, selectmode=SINGLE, font=fnt_1)
```

```
# Цвет фона и цвет для выделения пункта:
lst.configure(bg="gray96", selectbackground="gray")
# Способ выделения пункта и высота списка:
lst.configure(activestyle="none", height=len(fonts)+1)
# Заполнение статического списка пунктами:
for n in fonts:
    lst.insert(END, n)
# По умолчанию выбран первый пункт:
lst.select_set(0)
# Обработчик для статического списка:
lst.bind("<<ListboxSelect>>", setAll)
# Создание кнопки:
btn=Button(frm_btn, text="OK", font=fnt_2)
# Обработчик для кнопки:
btn.configure(command=wnd.destroy)
# Размещение меток и слайдера на панелях:
lbl_text.pack(side="top", fill="x", padx=5, pady=5)
lbl.pack(side="top", fill="both", padx=5, pady=5)
lbl_color.pack(side="top", fill="x", padx=5, pady=5)
scl.pack(side="bottom", fill="x", padx=5, pady=5)
lbl_size.pack(side="bottom", fill="x", padx=5, pady=[25,5])
lbl_font.pack(side="top", fill="x", padx=5, pady=5)
lbl_style.pack(side="top", fill="x", padx=5, pady=5)
# Размещение переключателей:
rb_1.pack(side="left", fill="x", padx=5, pady=5)
rb_2.pack(side="left", fill="x", padx=5, pady=5)
rb_3.pack(side="left", fill="x", padx=5, pady=5)
# Размещение опций:
chb_1.pack(side="left", fill="x", padx=5, pady=5)
chb_2.pack(side="left", fill="x", padx=5, pady=5)
# Размещение статического списка:
lst.pack(side="top", fill="x", padx=5, pady=5)
```

```
# Размещение кнопки:
btn.pack(side="bottom", fill="x", padx=50, pady=10)
# Размещение панелей:
frm_text.place(x=5, y=5, width=W-10, height=Hf)
frm_check.pack(side="bottom", fill="both", padx=5, pady=5)
frm_list.place(x=5, y=Hf+10, height=Hl, width=Wl)
frm_scale.place(x=Wl+10, y=Hf+10, width=Ws, height=Hs)
frm_btn.place(x=Wl+10, y=Hf+Hs+15, width=Ws, height=Hb)
# Применение параметров шрифта к шаблонному тексту:
setAll()
# Режим отслеживания значения переменных:
color.trace("w", setAll)
bold.trace("w", setAll)
italic.trace("w", setAll)
# Отображение окна:
wnd.mainloop()
```

Код объемный, но на самом деле достаточно простой. По большому счету, речь идет о выполнении операций трех типов:

- нам нужно создать (и настроить параметры) объекты для соответствующих графических компонентов;
- следует определить для элементов управления (то есть тех компонентов, которые изменяют свое состояние в процессе работы приложения) способ обработки событий;
- нужно разместить графические компоненты на панелях, а панели разместить в окне.

В программе объявляются два списка `fnt_1` и `fnt_2`, которые определяют шрифты, используемые для отображения текста в графических компонентах. Список `fonts` содержит названия шрифтов, которые будут отображаться в статическом списке в окне. Значения переменных `min_size` и `max_size` определяют соответственно минимальный и максимальный размер для шрифта, применяемого к шаблонному тексту. Также мы используем группу переменных, определяющих размеры

(ширину и высоту) некоторых графических компонентов, в том числе ширину и высоту главного окна (переменные W и H).



ПОДРОБНОСТИ

При создании объекта окна его размеры определяются командой `wnd.geometry(str(W)+"x"+str(H))`. Примечательно здесь то, что текст, определяющий ширину и высоту окна, мы сформировали объединением текстовых значений, в том числе используя значения переменных W и H.

Главное окно приложения создается как объект класса Tk, и ссылка на этот объект записывается в переменную wnd. Мы создаем пять объектов класса Frame (frm_scale, frm_text, frm_btn, frm_list и frm_check), с помощью которых реализуются панели. Все панели, за исключением frm_check (панель с опциями), добавляются в главное окно. Панель frm_check добавляется на панель frm_list (панель со статическим списком). Аргумент bd конструктора класса Frame определяет толщину рамки вокруг панели, а аргумент relief определяет ее стиль (значение GROOVE означает эффект «вдавленной» рамки).

Текстовые метки lbl_text, lbl_color, lbl_size, lbl_font и lbl_style создаются на основе класса Label и содержат неизменяемый текст. Это фактически подписи для элементов управления. Панель, на которую добавляется метка, указана первым аргументом конструктора класса Label. Несколько особо создается метка lbl, предназначенная для отображения шаблонного текста. Для этой метки задается белый цвет фона (инструкция bg="white"), а также определена рамка с эффектом «выдавливания» (инструкция relief=RAISED). Но есть еще одна важная инструкция textvariable=text, указанная в конструкторе класса Label. Эта инструкция связывает содержимое метки с содержимым «переменной» text, причем сама «переменная» создается командой text=StringVar() (конечно, на самом деле речь идет об объекте). Специфика ситуации в том, что если изменится значение «переменной» text, то автоматически изменится содержимое метки, а изменение содержимого метки приведет к автоматическому изменению «переменной» text. То есть в данном случае мы используем такой своеобразный механизм обработки событий: чтобы изменить содержимое метки lbl, достаточно изменить значение «переменной» text. Для присваивания «переменной» нового значения используется метод set(), а для считывания значения «переменной» используется метод get().

ПОДРОБНОСТИ

Еще раз подчеркнем, что, хотя мы говорим о «переменной» `text`, на самом деле речь, конечно, идет об объекте. Кроме текстовых «переменных», которые создаются на основе класса `StringVar`, существуют, например, классы `BooleanVar`, `DoubleVar` и `IntVar`, позволяющие создавать «переменные», аналогичные `text` и предназначенные для работы с данными других типов.

Аналогичная схема использована для работы с переключателями и опциями, для чего командами `color=StringVar()`, `bold=StringVar()` и `italic=StringVar()` создаются соответствующие «переменные».

НА ЗАМЕТКУ

Для некоторых компонентов настройка параметров выполняется вызовом метода `configure()` из объекта компонента, причем метод может вызываться несколько раз. Сделано это исключительно из эстетических соображений. На самом деле можно все аргументы, определяющие настройки компонента, задать в одной команде с вызовом метода `configure()` или задать эти параметры в команде создания компонента.

Переключатели `rb_1`, `rb_2` и `rb_3` реализуются как объекты класса `Radiobutton`. Первым аргументом конструктору передается ссылка на панель `frm_scale`, на которую будет добавляться переключатель. Аргумент конструктора `text` определяет надпись для соответствующего переключателя. Для всех трех переключателей для аргумента `variable` указана «переменная» `color`. Это означает, что, запросив значение `color` с помощью метода `get()`, мы можем получить значение переключателя, который установлен на данный момент. «Значения» переключателей задаются с помощью аргумента `value`. Мы в качестве значений используем англоязычные названия для цвета (значения `"red"`, `"blue"` и `"black"`). Аргумент `font` определяет шрифт, которым отображается подпись для переключателя.

В результате выполнения команды `color.set("blue")` устанавливается переключатель со значением `"blue"` (соответствует синему цвету) — поэтому при первом отображении окна в нем установлен переключатель выбора синего цвета.

Нечто похожее происходит при создании опций `chb_1` и `chb_2`. Обе опции добавляются на панель `frm_check`. Для опции `chb_1` значение

аргумента text равно "жирный", а для опции chb_2 значение аргумента text установлено "курсив". Это подписи, которые будут отображаться возле опций. Шрифт, которым эти надписи будут отображаться, определяется значением аргумента font.

Опция, как известно, может находиться в двух состояниях: у нее может быть установлен или отменен флажок. С каждым из этих состояний ассоциируется определенное значение. Значение, отождествляемое с состоянием с установленным флажком, определяется аргументом onvalue. Если флажок не установлен, то такое состояние отождествляется со значением, которое определяется аргументом offvalue. Узнать или задать эти значения можно с помощью «переменной», которая указана в качестве значения аргумента variable. Для обеих опций в качестве значения аргумента offvalue указан пустой текст. Значение аргумента onvalue для опции chb_1 равно "bold", а для опции chb_2 оно равно "italic". Опция chb_1 связана с «переменной» bold, а опция chb_1 связана с «переменной» italic. Поэтому при выполнении команд bold.set("") и italic.set("italic") для первой опции флажок отменяется, а для второй — устанавливается.

Объект слайдера scl создается на основе класса Scale. Слайдер добавляется на панель frm_scale. Значение HORIZONTAL для аргумента orient означает, что слайдер ориентирован горизонтально.



ПОДРОБНОСТИ

Кроме значения HORIZONTAL можно использовать значение VERTICAL, означающее, что слайдер ориентирован вертикально. Также при создании объекта слайдера можно задать значение для аргумента label, которое будет определять подпись для слайдера. Для горизонтально расположенного слайдера она располагается в левом верхнем углу, а для вертикально расположенного слайдера — в правом верхнем углу. Мы вместо этого используем отдельную метку.

Диапазон изменения значений на слайдере определяется с помощью аргументов from_ и to. Аргумент tickinterval определяет интервал, с которым отображаются метки (подписей) на шкале слайдера, а аргумент resolution определяет шаг дискретности для изменения положения ползунка. В качестве обработчика для события, связанного с изменением положения ползунка на слайдере, определяется функция setAll() (имя функции присваивается в качестве значения аргументу command).



ПОДРОБНОСТИ

В результате вызова функции `setAll()` считываются настройки всех элементов управления в окне, и в соответствии с этими настройками формируется и отображается нужным шрифтом шаблонный текст. Наша «стратегия» состоит в том, чтобы при изменении состояния любого из элементов управления (статический список, опции, переключатели или слайдер) вызывалась функция `setAll()`. При этом речь идет об обработке разных событий, и в некоторых случаях при вызове функции-обработчика ей передаются аргументы (например, ссылка на объект события). Мы эти аргументы в процессе обработки событий не используем, но они могут быть, и шаблон функции должен соответствовать формату, в котором функция будет вызываться. Поэтому мы применяем искусственный прием — в описании функции указан аргумент со звездочкой `*`. Это означает, что функции при вызове может передаваться произвольное количество аргументов. Более детально код функции `setAll()` анализируется немного позже.

Статический список `lst` создается на основе класса `Listbox`. Статический список добавляется на панель `frm_list`. Значение `SINGLE` аргумента `selectmode` означает, что в списке можно выбрать только один пункт.



ПОДРОБНОСТИ

Возможные значения аргумента `selectmode`: `SINGLE` (возможен выбор одного пункта), `BROWSE` (возможен выбор одного пункта с возможностью перетаскивания выделения мышью), `MULTIPLE` (возможен выбор нескольких пунктов) и `EXTENDED` (возможен выбор нескольких пунктов, причем они могут быть в разных местах списка).

Аргумент `bg` определяет цвет фона для статического списка. Аргумент `selectbackground` задает цвет, которым выделяется выбранный пункт. Значения `"gray96"` и `"gray"` соответствуют разным оттенкам серого цвета.



ПОДРОБНОСТИ

Среди текстовых констант, определяющих разные цвета, есть группа констант, определяющих оттенки серого цвета. Название констант получается объединением текста `"gray"` и текстового представления для числа от `"0"` до `"100"`. Константа `"gray0"` соответствует черному цвету, а константа `"gray100"` соответствует белому цвету. Вместо текста `"gray"` можно также использовать текст `"grey"`.

Аргумент `height` определяет высоту статического списка (в строках). Значение `len(fonts)+1` означает, что статически список содержит на одну строку больше, чем пунктов в этом списке (поскольку количество пунктов в статическом списке определяется количеством элементов в списке `fonts`). Также мы для аргумента `activestyle` указываем значение `"none"`. Это означает, что пункт в статическом списке при выборе будет выделяться только цветом.



ПОДРОБНОСТИ

Возможные значения для аргумента `activestyle`: `"dotbox"` (вокруг выбранного пункта списка отображается пунктирная рамка), `"underline"` (выбранный пункт выделяется подчеркиванием) и `"none"` (дополнительные эффекты не используются).

После создания статического списка его необходимо заполнить (добавить в него пункты). Для этого запускается оператор цикла, в котором перебирается содержимое списка `fonts`. За каждый цикл выполняется команда `lst.insert(END, n)`, которой в конец статического списка добавляется очередной элемент `n` из списка `fonts`.



НА ЗАМЕТКУ

Поскольку первым аргументом метода `insert()` указано значение `END`, но соответствующий элемент (второй аргумент метода) добавляется в конец статического списка. Вообще же первым аргументом можно передавать индекс, определяющий позицию, на которую в статическом списке помещается добавляемый элемент.

Выполнение команды `lst.select_set(0)` означает, что по умолчанию в статическом списке будет выбран элемент с нулевым индексом. Наконец, командой `lst.bind("<<ListboxSelect>>", setAll)` для статического списка задается обработчик события `<<ListboxSelect>>`, связанного с выбором пункта в списке. В этом случае вызывается функция `setAll()`.

Кнопка, предназначенная для закрытия окна, создается уже знакомым нам способом на основе класса `Button`.

После создания объектов для графических компонентов они размещаются на панелях, а затем панели размещаются в окне. Как отмечалось ранее, для размещения компонентов мы используем метод `pack()`.

Аргумент `side` этого метода определяет место, в которое добавляется компонент. Возможные значения этого аргумента: `"top"` (размещение вверху), `"bottom"` (размещение внизу), `"left"` (размещение слева) и `"right"` (размещение справа). Аргумент `fill` определяет режим масштабирования компонента. Значение `"both"` для аргумента означает, что компонент в случае необходимости масштабируется и вдоль горизонтали, и вдоль вертикали. Значение `"x"` означает, что масштабирование выполняется вдоль горизонтали. Значение `"y"` используется, если масштабирование должно выполняться вдоль вертикали. Значение `"none"` соответствует ситуации, когда масштабирование не применяется.

Аргументы `padx` и `pady` определяют внешние отступы (в пикселях) по отношению к смежным компонентам соответственно вдоль горизонтали и вертикали. Если значением аргумента указан список из двух элементов, то они определяют отступы с одной и другой стороны компонента.



НА ЗАМЕТКУ

Команды, использованные для размещения компонентов, все однотипные и комментариев не требуют. Единственное, на что хочется обратить внимание, это то, что важен порядок, в котором компоненты добавляются в контейнер. Например, если мы последовательно добавляем два компонента и в обоих случаях в качестве значения аргумента `side` указано `"left"`, то компоненты будут добавляться в левую часть контейнера один за другим. Если для обоих указано значение `"top"`, то они добавляются один под другим. Если для первого указано значение `"left"`, а для другого `"top"`, то в левую часть контейнера добавляется первый компонент, а второй добавляется в верхнюю часть свободной области контейнера. Если поменять порядок добавления компонентов, то сначала будет добавлен компонент в верхнюю часть контейнера, а затем в левую часть свободной области добавляется следующий компонент. Это особенно важно, если используется масштабирование компонентов.

Для размещения панелей в окне использован уже знакомый нам метод `place()`. Достойно внимания здесь то, что для вычисления положения и размеров (ширины и высоты) компонентов мы используем созданные ранее переменные. Такой подход удобен тем, что если нам впоследствии понадобится изменить размеры того или иного компонента, то, скорее всего, все сведется к изменению значения соответствующей переменной.

ПОДРОБНОСТИ

Напомним, что при работе с методом `place()` положение компонента определяется с помощью аргументов `x` и `y`, а его размеры (в пикселях) задаются с помощью аргументов `width` (ширина) `height` (высота).

После того как все компоненты размещены, вызывается функция `setAll()`, в результате чего в соответствии с настройками управляющих компонентов вычисляются параметры шрифта для шаблонного текста, определяется сам шаблонный текст и этот текст (с использованием соответствующего шрифта) отображается в окне. Также перед отображением окна командой `wnd.mainloop()`, командами `color.trace("w", setAll)`,`bold.trace("w", setAll)` и `italic.trace("w", setAll)` для «переменных» `color`, `bold` и `italic` устанавливается режим отслеживания значения: каждый раз, когда будет изменяться значение соответствующей переменной, будет вызываться функция `setAll()`. Данными командами фактически определяется способ обработки событий, связанных с изменением состояния переключателей и опций.

ПОДРОБНОСТИ

Первым аргументом методу `trace()` можно передавать такие значения: `"w"` (отслеживается изменение значения «переменной"), `"r"` (отслеживается считывание значения «переменной") и `"u"` (отслеживается ситуация, связанная с удалением «переменной").

Необходимость в отслеживании значений «переменных» связана с тем, что эти значения могут происходить вследствие манипуляций пользователя с соответствующими элементами управления. Мы хотим, чтобы в таких случаях вызывалась функция `setAll()`. Имя этой функции передается вторым аргументом методу `trace()`.

Команды с вызовом метода `trace()` размещены в конце программного кода, перед командой отображения окна, чтобы при настройке параметров графических компонентов, влияющих на значения «переменных» `color`, `bold` и `italic`, не вызывалась функция `setAll()`.

Нам осталось проанализировать код двух функций, которые использованы в программе, причем одна из них вызывается в другой.

Функция `getFont()` предназначения для определения шрифта по настройкам компонентов окна. Результатом она возвращает список, определяющий название, размер и стиль шрифта. Результат функции

вычисляется следующим образом. Сначала создается пустой список `res`. Командой `name=lst.get(lst.curselection())` определяется название шрифта. Для этого мы из объекта статического списка `lst` вызываем метод `get()`. Метод возвращает в качестве результата название пункта в статическом списке с индексом, указанным аргументом метода. Значением выражения `lst.curselection()` является индекс выделенного в статическом списке элемента.

Размер шрифта вычисляется командой `size=scl.get()`. В данном случае мы из объекта слайдера `scl` вызываем метод `get()`, который в качестве результата возвращает значение, установленное на слайдере. Командами `res.append(name)` и `res.append(size)` название и размер шрифта добавляются в список `res`. После этого начинается определение стиля шрифта. В условном операторе проверяется условие `bold.get()!=""`. Оно истинно, если у опции применения жирного шрифта установлен флажок. Если так, то командой `res.append(bold.get())` в список `res` добавляется значение `"bold"`.



ПОДРОБНОСТИ

Стоит напомнить, что для опции выбора жирного стиля установлены два значения: пустой текст `""`, если флажок опции не установлен, и текст `"bold"`, если флажок установлен. Также с опцией связана «переменная» `bold`, позволяющая определить состояние опции. Чтобы прочитать значение (определяющее состояние опции), используется инструкция `bold.get()`. Если флажок опции не установлен, то значением выражения `bold.get()` является пустой текст `""`. Если флажок опции установлен, то значением выражения `bold.get()` является текст `"bold"`.

Это же замечание относится и к определению состояния опции, предназначенной для применения курсивного стиля. Следует лишь сделать поправку на то, что с этой опцией связана «переменная» `italic`, а возможными значениями выражения `italic.get()` является пустой текст `""` (флажок опции не установлен) или текст `"italic"` (флажок опции установлен).

Если установлена опция применения курсивного стиля, то истинным является условие `italic.get()!=""` и командой `res.append(italic.get())` в список res добавляется элемент `"italic"`. По завершении вычислений ссылка на список `res` возвращается в качестве результата функции.

Функция `setAll()` предназначена для применения параметров шрифта. Как отмечалось ранее, для того чтобы функцию можно было вызывать с разными аргументами (которые мы не собираемся использовать), она описана как такая, что может иметь произвольное количество аргументов (аргумент `*args` в описании функции). В теле функции командой `fnt=getFont()` на основе настроек графических компонентов вычисляется шрифт, и ссылка на список с параметрами шрифта записывается в переменную `fnt`. Командой `lbl.configure(font=fnt)` к метке отображения шаблонного текста применяется шрифт, вычисленный функцией `getFont()`. Командой `lbl.configure(fg=color.get())` задается цвет для отображения текста. Для этого аргументу `fg` в качестве значения присваивается выражение `color.get()`. Здесь мы учли, что «переменная» `color` связана с группой переключателей, и при вызове метода `get()` мы получаем значение, связанное с переключателем, который в данный момент установлен. Поскольку при создании переключателей с каждым из них мы ассоциировали текстовое название соответствующего цвета, то результатом выражения `color.get()` является название цвета, что нам и нужно.

После этого начинается формирование собственно шаблонного текста. Переменная `txt` получает начальное значение `"\nШрифт "`, и затем это значение «уточняется». Сначала командой `txt+=fnt[0]` к текстовой строке дописывается название шрифта, которое является первым элементом в списке `fnt`. Также в текст добавляется информация о размере шрифта (второй элемент `fnt[1]` в списке `fnt`). С помощью условных операторов мы проверяем, есть ли в списке `fnt` элементы `"bold"` (жирный стиль) и `"italic"` (курсивный стиль), и если элемент в списке представлен, то к текстовой строке дописывается соответствующий текстовый фрагмент. После этого в зависимости от цвета шрифта к текстовой строке `txt` добавляется и такая информация. После того как текст сформирован, командой `text.set(txt)` он применяется для метки `lbl` с шаблонным текстом. Здесь мы учли, что с этой меткой связана «переменная» `text`. Применение нового значения для «переменной» с помощью метода `set()` приводит к автоматическому обновлению содержимого метки.

Использование меню

— По-вашему, это не интересно?
— Интересно. Для любителей древности.

Из к/ф «Приключения Шерлока Холмса и доктора Ватсона»

Следующая программа, которую мы рассмотрим, является вариацией предыдущего примера. Как и ранее, мы будем с помощью элементов управления определять параметры шрифта, а в области окна приложения с применением этого шрифта будет отображаться шаблонный текст. Но теперь вместо статического списка, слайдера, опций и переключателей мы будем использовать *панель меню* и *панель инструментов*. Окно, которое будет появляться при запуске программы на выполнение, показано на рис. 11.13.

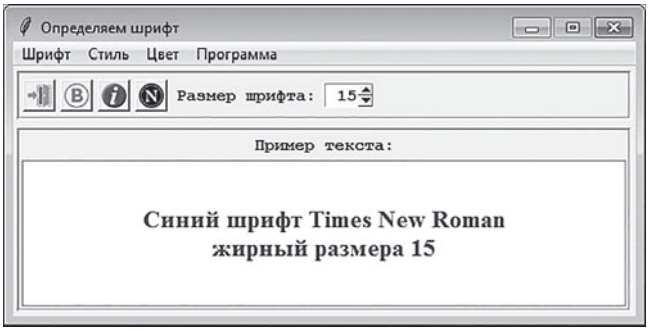


Рис. 11.13. Окно приложения с меню и панелью инструментов

Окно содержит панель меню с пунктами **Шрифт**, **Стиль**, **Цвет** и **Программа**. Под панелью меню расположена панель инструментов с четырьмя кнопками, метка с текстом и *спиннер*, в котором можно выбирать размер шрифта (допустимые значения находятся в диапазоне от **15** до **20**). Под панелью инструментов располагается еще одна вспомогательная панель с подписью и шаблонным текстом на белом фоне. Текст содержит информацию о названии, стиле, размере и цвете шрифта. По умолчанию используется шрифт **Times New Roman**, жирный, размера **15**, синего цвета. Выбрать новый шрифт можно в пункте меню **Шрифт**, как показано на рис. 11.14.

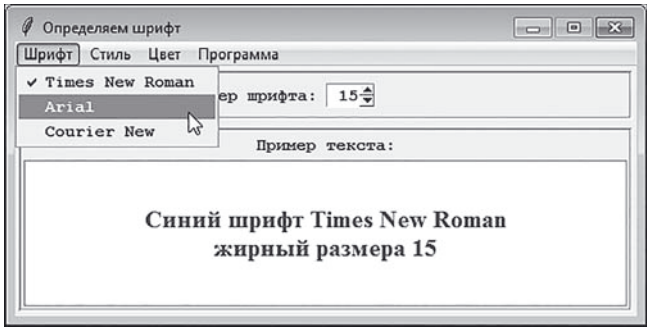


Рис. 11.14. Выбор шрифта в меню **Шрифт**

В этом пункте меню используются команды типа переключателей: возле выбранного пункта отображается флажок, и этот флажок один и только один. При выборе нового шрифта (или изменении любого иного параметра шрифта) он применяется к шаблонному тексту, причем сам текст тоже меняется.

На рис. 11.15 проиллюстрировано содержимое пункта меню **Стиль**, предназначенного для установки стиля шрифта.

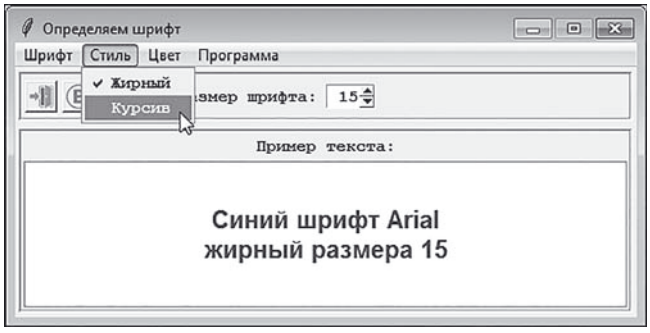


Рис. 11.15. Выбор стиля шрифта в меню **Стиль**

В пункте меню **Стиль** две команды (для установки/отмены жирного и/или курсивного стиля), причем это команды опционного типа: возле каждой из них, независимо от другой, может быть установлен или отменен флажок.

В пункте меню **Цвет**, содержимое которого представлено на рис. 11.16, можно выбрать цвет для отображения текста.

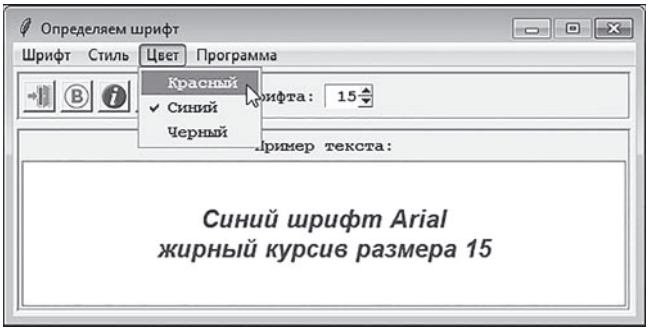


Рис. 11.16. Выбор цвета шрифта в меню **Цвет**

Три команды для выбора цвета также относятся к командам типа переключателей: возле одной из этих команд отображается флажок.

Наконец, в пункте меню **Программа** всего две команды (между которыми расположен разделитель). Содержимое этого пункта меню можно видеть на рис. 11.17.

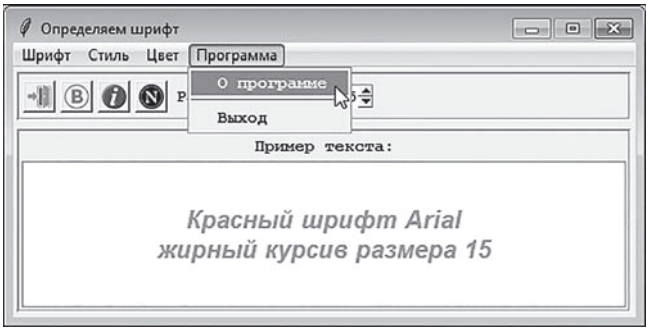


Рис. 11.17. Содержимое меню **Программа**

Там всего две команды. При выборе команды **Выход** приложение завершает работу. Если выбрать команду **О программе**, то на фоне окна приложения будет открыто модальное окно с информацией о программе, как показано на рис. 11.18.

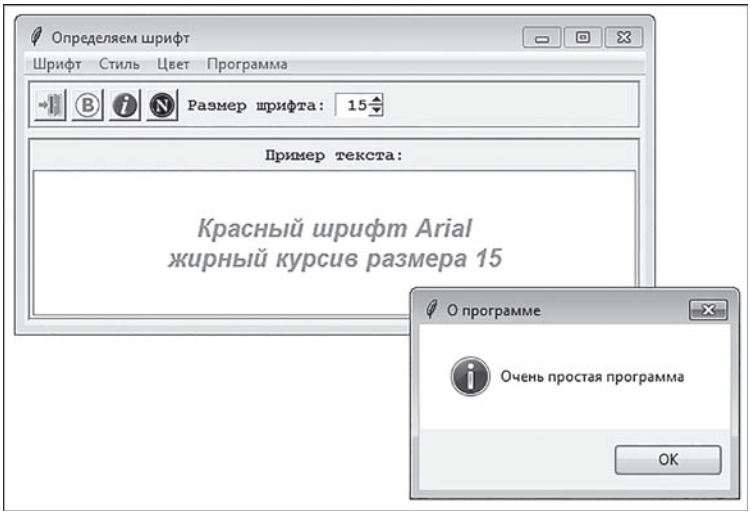


Рис. 11.18. Окно с сообщением на фоне окна приложения

Пока не будет закрыто модальное окно, основное окно будет заблокировано.

Также в приложении используется *контекстное меню*: если в рабочей области приложения нажать правую кнопку мыши, то появится контекстное меню с командами, позволяющими выбрать шрифт, стиль, цвет, или завершить работу с приложением. Открытое контекстное меню представлено на рис. 11.19.

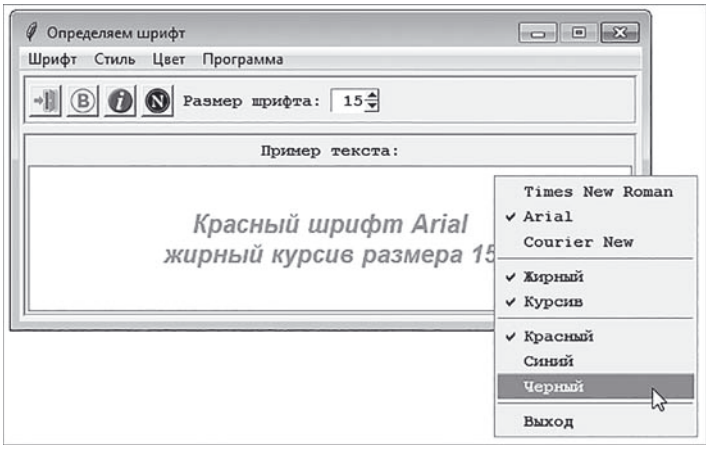


Рис. 11.19. Выбор цвета в контекстном меню

Следует отметить, что контекстное меню полностью синхронизировано с командами в главном меню — например, если в контекстном меню выбрать цвет, то информация об этом выборе отобразится и в главном меню, и наоборот.

Размер шрифта можно изменить с помощью спиннера, как показано на рис. 11.20.

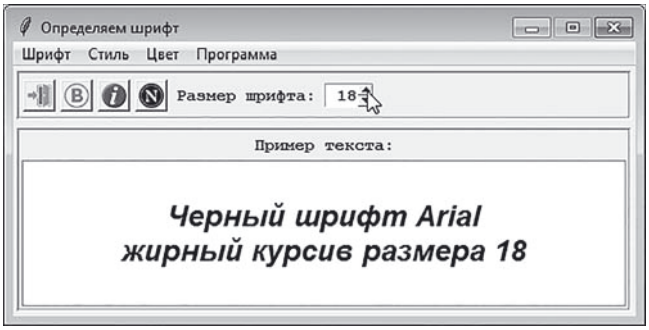


Рис. 11.20. Изменение размера шрифта с помощью спиннера

Панель инструментов содержит, кроме спиннера, еще четыре кнопки. При нажатии первой кнопки слева окно закрывается и приложение завершает работу. Следующие две кнопки (с символами **B** и **i**) предназначены для применения/отмены соответственно жирного и курсивного стилей. Нажатие кнопки приводит к изменению настроек на «противоположные». Например, если был установлен жирный стиль, то нажатие кнопки с символом **B** приведет к отмене жирного стиля. Если жирный стиль не был установлен, то нажатие кнопки приведет к применению жирного стиля. Кнопка с литерой **N** предназначения для отмены жирного и курсивного стиля: при нажатии кнопки стиль будет обычным (не жирным и не курсивным). На рис. 11.20 проиллюстрированы последствия нажатия кнопки отмены жирного и курсивного стилей.

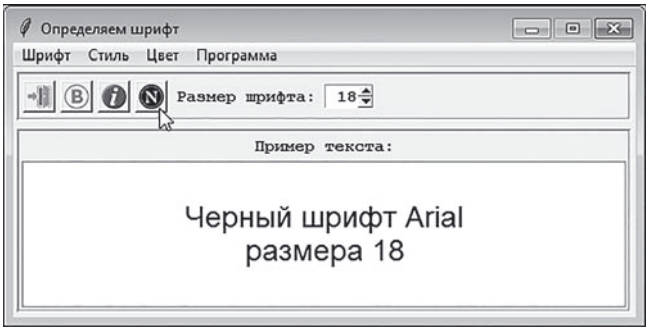


Рис. 11.21. Результат нажатия кнопки отмены жирного и курсивного стилей

Так выглядит и функционирует приложение. Теперь проанализируем соответствующий программный код, представленный в листинге 11.6.

Листинг 11.6. Работа с меню

```
from tkinter import *
from tkinter.messagebox import *
# Класс для создания окна:
class MyApp:
    # Конструктор:
    def __init__(self):
        # Настройка параметров:
        self.setValues()
        # Создание главного окна:
        self.makeMainWindow()
        # Определение "переменных" для обработки событий:
        self.setVars()
        # Создание главного меню:
        self.makeMainMenu()
        # Создание панели инструментов:
        self.makeToolBar()
        # Создание вспомогательной панели:
        self.makeFrame()
        # Создание контекстного меню:
        self.makePopupMenu()
        # Вычисление параметров шаблонного текста:
        self.apply()
        # Режим отслеживания "переменных":
        self.traceVars()
        # Отображение главного окна:
        self.showMainWindow()
    # Метод для настройки параметров:
    def setValues(self):
        # Ширина и высота окна:
```

```
        self.W=500
        self.H=200
        # Высота панели инструментов:
        self.h=40
        # Размеры главного окна:
        self.position=str(self.W)+"x"+str(self.H)
        # Названия шрифтов:
        self.fonts=["Times New Roman","Arial","Courier New"]
        # Словарь с названиями цвета:
        self.colors={"red":"Красный","blue":"Синий", "black":"Черный"}
        # Список с названиями стиля:
        self.style=[["bold","Жирный"],["italic","Курсив"]]
        # Список с названиями файлов с изображениями:
        self.imgFiles=["exit.png","bold.png","italic.png", "normal.png"]
        # Путь к каталогу с файлами:
        self.path="D:\\Books\\Python\\Pictures\\"
        # Основной шрифт:
        self.font=("Courier New",10,"bold")
    # Метод для создания главного окна:
    def makeMainWindow(self):
        self.wnd=Tk()
        self.wnd.title("Определяем шрифт")
        self.wnd.geometry(self.position)
        self.wnd.resizable(False, False)
    # Метод для отображения главного окна:
    def showMainWindow(self):
        self.wnd.mainloop()
    # Метод для создания главного меню:
    def makeMainMenu(self):
        # Создание объекта для панели меню:
        self.menubar=Menu(self.wnd)
        # Создание пунктов главного меню:
```

```
        self.mFont=Menu(self.wnd, font=self.font, tearoff=0)
        self.mStyle=Menu(self.wnd, font=self.font, tearoff=0)
        self.mColor=Menu(self.wnd, font=self.font, tearoff=0)
        self.mAbout=Menu(self.wnd, font=self.font, tearoff=0)
        # Заполнение пунктов меню:
        self.setMenuFont(self.mFont)
        self.setMenuStyle(self.mStyle)
        self.setMenuColor(self.mColor)
        # Заполнение последнего пункта меню:
        self.mAbout.add_command(label="О программе", command=self.showDialog)
        # Добавление разделителя:
        self.mAbout.add_separator()
        self.mAbout.add_command(label="Выход", command=self.clExit)
        # Добавление пунктов меню на панель меню:
        self.menubar.add_cascade(label="Шрифт", menu=self.mFont)
        self.menubar.add_cascade(label="Стиль", menu=self.mStyle)
        self.menubar.add_cascade(label="Цвет", menu=self.mColor)
        self.menubar.add_cascade(label="Программа", menu=self.mAbout)
        # Задаем главное меню для окна:
        self.wnd.config(menu=self.menubar)
    # Метод для создания панели инструментов:
    def makeToolBar(self):
        # Список с названиями методов для обработки
        # события, связанного с нажатием кнопок:
        mt=[self.clExit, self.clBold, self.clItalic, self.clNormal]
        # Панель для размещения кнопок:
        self.toolbar=Frame(self.wnd, bd=3, relief=GROOVE)
        # Список для изображений:
        self.imgs=[]
        # Список для кнопок:
        self.btns=[]
        # Создание изображений и кнопок:
```

```
        for f in self.imgFiles:
            # Создание изображения:
            self.imgs.append(PhotoImage(file=self.path+f))
            # Создание кнопки:
            self.btns.append(Button(self.toolbar, image=self.imgs[-1]))
            # Добавление кнопки на панель:
            self.btns[-1].pack(side="left", padx=2, pady=2)
        # Определение обработчиков для кнопок:
        for k in range(len(mt)):
            self.btns[k].configure(command=mt[k])
        # Создание текстовой метки:
        self.lblSize=Label(self.toolbar, text="Размер шрифта:", font=self.font)
        # Размещение метки на панели:
        self.lblSize.pack(side="left", padx=2, pady=2)
        # Создание спиннера:
        self.spnSize=Spinbox(self.toolbar, from_=15, to=20, font=self.font,
                 width=3, justify="right", textvariable=self.size)
        # Размещение спиннера на панели:
        self.spnSize.pack(side="left", padx=2, pady=2)
        # Размещение панели в окне:
        self.toolbar.place(x=3, y=3, width=self.W-6, height=self.h)
    # Метод для создания вспомогательной панели:
    def makeFrame(self):
        # Создание вспомогательной панели:
        self.frame=Frame(self.wnd, bd=3, relief=GROOVE)
        # Создание метки и добавление ее на панель:
        Label(self.frame, text="Пример текста:", font=self.font).pack(side="top")
        # Создание метки для отображения
        # шаблонного текста:
        self.lblText=Label(self.frame, textvariable=self.text, relief=GROOVE,
                bg="white", height=5)
        # Размещение метки на вспомогательной панели:
```

```
        self.lblText.pack(side="top", fill="both", padx=1, pady=1)
        # Размещение вспомогательной панели в окне:
        self.frame.place(x=3, y=self.h+9, width=self.W-6, height=self.H-self.h-12)
    # Метод для создания контекстного меню:
    def makePopupMenu(self):
        # Создание объекта контекстного меню:
        self.popup=Menu(self.wnd, tearoff=0, font=self.font)
        # Добавление команд в контекстное меню:
        self.setMenuFont(self.popup)
        # Добавление разделителя:
        self.popup.add_separator()
        # Добавление команд в контекстное меню:
        self.setMenuStyle(self.popup)
        # Добавление разделителя:
        self.popup.add_separator()
        # Добавление команд в контекстное меню:
        self.setMenuColor(self.popup)
        # Добавление разделителя:
        self.popup.add_separator()
        # Добавление команды в контекстное меню:
        self.popup.add_command(label="Выход", command=self.clExit)
        # Определение обработчика события
        # для контекстного меню:
        self.wnd.bind("<Button-3>", lambda evt: self.popup.tk_popup(evt.x_root,
                      evt.y_root))
    # Метод для формирования команд меню,
    # связанных с выбором шрифта:
    def setMenuFont(self, menu):
        for f in self.fonts:
            menu.add_radiobutton(label=f, value=f, variable=self.name)
        self.name.set(self.fonts[0])
    # Метод для формирования команд меню,
```

```
    # связанных с выбором стиля:
    def setMenuStyle(self, menu):
        for k in range(len(self.style)):
            menu.add_checkbutton(label=self.style[k][1], onvalue=True,
                    offvalue=False, variable=self.bi[k])
        self.bi[0].set(True)
        self.bi[1].set(False)
    # Метод для формирования команд меню,
    # связанных с выбором цвета:
    def setMenuColor(self, menu):
        for r in self.colors.keys():
            menu.add_radiobutton(label=self.colors[r], value=r,
                    variable=self.color)
        self.color.set("blue")
    # Метод для определения параметров шрифта и
    # вычисления шаблонного текста:
    def apply(self,*args):
        # Цвет для шрифта:
        clr=self.color.get()
        # Название шрифта:
        nm=self.name.get()
        # Размер шрифта:
        sz=self.size.get()
        # Применение цвета к метке:
        self.lblText.configure(fg=clr)
        # Список с параметрами шрифта:
        fnt=[nm, sz]
        # Формирование шаблонного текста и
        # определение параметров шрифта:
        txt=self.colors[clr]+" шрифт "+nm+"\n"
        for k in range(len(self.style)):
            if self.bi[k].get():
```

```
            fnt.append(self.style[k][0])
            txt+=self.style[k][1].lower()+" "
        txt+="размера "+str(sz)
        # Применение шрифта к метке:
        self.lblText.configure(font=fnt)
        # Задаем текст для метки:
        self.text.set(txt)
    # Метод для создания "переменных", используемых
    # для обработки событий:
    def setVars(self):
        self.text=StringVar()
        self.name=StringVar()
        self.bi=[BooleanVar(), BooleanVar()]
        self.size=IntVar()
        self.color=StringVar()
    # Метод для перехода в режим отслеживания
    # значений "переменных":
    def traceVars(self):
        mt=self.apply
        self.name.trace("w", mt)
        self.color.trace("w", mt)
        for k in range(len(self.bi)):
            self.bi[k].trace("w", mt)
        self.size.trace("w", mt)
    # Метод для обработки нажатия кнопки
    # для завершения работы:
    def clExit(self):
        self.wnd.destroy()
    # Метод для обработки нажатия кнопки
    # применения/отмены жирного стиля:
    def clBold(self):
        self.bi[0].set(not self.bi[0].get())
```

```
    # Метод для обработки нажатия кнопки
    # применения/отмены курсивного стиля:
    def clItalic(self):
        self.bi[1].set(not self.bi[1].get())
    # Метод для обработки нажатия кнопки
    # отмены жирного и курсивного стиля:
    def clNormal(self):
        self.bi[0].set(False)
        self.bi[1].set(False)
    # Метод для отображения окна с сообщением:
    def showDialog(self):
        showinfo("О программе","Очень простая программа")
# Отображение окна на экране:
MyApp()
```

В этой программе мы немного изменили общий стратегический подход, а именно, в ней описывается класс `MyApp`, а затем на его основе создается объект (анонимный, поскольку ссылка на этот объект никуда не записывается). Ссылки на объекты графических компонентов (и другие важные характеристики) реализуются как поля объекта класса. Также для удобства восприятия мы разбили выполняемый код на отдельные блоки, реализованные как методы объекта.

При создании объекта класса вызывается, как известно, конструктор. Конструктор класса `MyApp` описан так, что в нем вызывается несколько других методов. Сначала вызывается метод `setValues()`, с помощью которого задаются все основные параметры объекта, используемые в работе приложения. В частности, при вызове метода создаются:

- поля `W` и `H` объекта с данными о ширине и высоте главного окна;
- поле `h` со значением высоты панели инструментов;
- текстовое поле `position` с размерами окна (для передачи в качестве аргумента методу `geometry()`);
- поле-список `fonts` с названиями шрифтов;
- поле-словарь `colors` с названиями цвета (ключом является английское название, а значение — русское название);

- поле-список из двух элементов `style`, и каждый его элемент содержит английское и русское название для стиля;
- поле-список `imgFiles` с названиями файлов с изображениями (используется при создании кнопок на панели инструментов);
- текстовое поле `path`, определяющее путь к каталогу с файлами изображений;
- поле-кортеж `font`, определяющее параметры шрифта, используемого в окне для отображения подписей.

Главное окно создается вызовом метода `makeMainWindow()`, при этом ссылка на созданный объект окна записывается в поле `wnd`.

С помощью метода `setVars()` создаются «переменные», которые мы планируем использовать в процессе обработки событий. Каждая такая «переменная» (`text`, `name` и `color` класса `StringVar`, `size` класса `IntVar`, а также поле-список `bi` с двумя «переменными» класса `BooleanVar`) реализована как поле объекта.

Для создания главного меню вызывается метод `makeMainMenu()`. Команды в теле этого метода требуют некоторого внимания. Так, сначала командой `self.menubar=Menu(self.wnd)` создается объект для панели меню. Объект создается на основе класса `Menu`. Аргументом указана ссылка на окно, в которое будет добавляться панель меню. Пункты для главного меню (`mFont`, `mStyle`, `mColor` и `mAbout`) также создаются на основе класса `Menu`. В качестве аргументов конструктору передаются ссылка на объект окна и шрифт, а аргумент `tearoff` со значением `0` означает, что пункт меню нельзя будет открепить главного меню.

> **НА ЗАМЕТКУ**
>
> По умолчанию при раскрытии пункта меню непосредственно под ним отображается пунктирная линия, щелчок на которой приводит к откреплению содержимого пункта и отображению этого содержимого в отдельной панели.

После создания команд меню их необходимо добавить в соответствующий пункт меню. Для решения этой задачи мы используем специально созданные методы `setMenuFont()`, `setMenuStyle()` и `setMenuColor()`, а последний пункт заполняем в явном виде.



НА ЗАМЕТКУ

В качестве аргументов методам `setMenuFont()`, `setMenuStyle()` и `setMenuColor()` передаются, соответственно, ссылки `mFont`, `mStyle` и `mColor` на заполняемые пункты меню. Сами методы и необходимость передачи им упомянутых аргументов обусловлены тем, что мы используем эти же методы при формировании содержимого контекстного меню. Методы описываются далее.

Для добавления команд в пункт меню `mAbout` из объекта пункта меню вызывается метод `add_command()`. Аргумент `label` определяет надпись (название команды), аргумент `command` определяет метод, вызываемый при выборе команды. Для добавления разделителя (линии) между командами используется метод `add_separator()`.



НА ЗАМЕТКУ

При вызове метода `showDialog()` отображается модальное окно справки. При вызове метода `clExit()` закрывается окно приложения. Методы описываются немного позже.

Для добавления пунктов в главное меню используется метод `add_cascade()`, который вызывается из объекта главного меню `menubar`. Аргумент `label` определяет название пункта меню, а аргумент `menu` содержит ссылку на объект добавляемого пункта меню.

После того как главное меню сформировано и заполнено, из объекта окна `wnd` вызывается метод `config()` в качестве аргумента `menu` которому передается ссылка `menubar` на объект главного меню.

Теперь проанализируем методы, использованные для формирования команд для пунктов меню. Второй аргумент метода `setMenuFont()` (как и прочих методов) называется `menu`. Мы исходим из того, что это ссылка на объект пункта меню, который заполняется. В операторе цикла переменная `f` принимает значения из списка `fonts` (названия шрифтов). Из объекта заполняемого пункта `menu` вызывается метод `add_radiobutton()`, в пункт меню добавляется команда типа переключателя. Значение `f` для аргументов `label` и `value` означает, что название шрифта будет отображаться как название команды, и оно же будет значением «переменной» `name`, ссылка на которую указана в качестве значения аргумента `variable`, в том случае, если выбрана соответствующая команда. После завершения оператора цикла командой

`self.name.set(self.fonts[0])` задается значение «переменной» `name` — это название первого шрифта в списке `fonts` (и соответствующая команда будет выделена флажком в пункте меню).

Прочие методы реализованы аналогично. В методе `setMenuStyle()` с помощью индекса `k` перебираются элементы списка `style`. Для добавления команды опционного типа используется метод `add_checkbutton()`. Отображаемое русское название стиля (аргумент `label`) определяется элементом `style[k][1]`. С опционными командами ассоциируются логические «переменные» — элементы поля-списка `bi` (инструкция `variable=self.bi[k]`). При установленном флажке для команды соответствующая логическая «переменная» принимает значение `True` (значение аргумента `onvalue`), а если флажок не установлен, то значение «переменной» равно `False` (значение аргумента `offvalue`). Командами `self.bi[0].set(True)` и `self.bi[1].set(False)`, задаются значения элементов из списка `bi`, что соответствует установленному флажку первой опции (опция выбора жирного стиля) и отсутствию флажка у другой опции (опция выбора курсивного стиля).

В теле метода `setMenuColor()` перебираются (с помощью переменной `r`) ключи словаря `colors`. В пункт меню с помощью метода `add_radiobutton()` добавляются команды типа переключателя. Название команды определяется значением элемента из словаря `colors` с ключом `r`. Ключ `r` является значением «переменной» `color` — все это благодаря инструкциям `value=r` и `variable=self.color`.



НА ЗАМЕТКУ

Получается, что отображается русское название текста, а при выборе команды значением «переменной» `color` будет английское название текста.

С помощью команды `self.color.set("blue")` мы задаем значение `"blue"` для «переменной» `color` (поэтому по умолчанию будет выбран синий цвет для отображения текста).

Панель инструментов создается при вызове метода `makeToolBar()`. В теле метода командой `mt=[self.clExit, self.clBold, self.clItalic, self.clNormal]` создается список с названиями методов, предназначенных для обработки события, связанного с нажатием кнопок на панели инструментов.

НА ЗАМЕТКУ

Методы указаны в том порядке, в котором кнопки будут добавляться на панель. И это тот же порядок, в котором в списке `imgFiles` указаны названия файлов с изображениями, предназначенными для размещения на кнопках.

Панель инструментов создается как обычная панель — объект класса `Frame`. Ссылка на панель записывается в поле `toolbar`. Кроме этого, мы создаем еще два поля-списка: `imgs` для записи в него ссылок на объекты изображений, а также `btns` для запоминания ссылок на объекты кнопок. Изображения и кнопки создаются в операторе цикла, в котором переменная `f` принимает значения из списка `imgFiles` (названия файлов с изображениями). Объекты изображений создаются на основе класса `PhotoImage` и добавляются в список `imgs` с помощью метода `append()`.

Кнопки создаются на основе класса `Button`, причем значением аргумента `image` указана ссылка на последнее (с индексом `-1`) на данный момент изображение в списке `imgs`. Соответствующая картинка будет отображаться на кнопке.

НА ЗАМЕТКУ

Изображения и кнопки создаются и добавляются в списки синхронно. Поэтому добавляемой (на определенном этапе) в список `btns` кнопке соответствует последнее добавленное в список `imgs` изображение.

Кнопки на панель добавляются с помощью метода `pack()`. При этом ссылку на последнюю (только что добавленную в список `btns`) кнопку можно получить через инструкцию `btns[-1]`.

После добавления кнопок на панель они заново перебираются, и для каждой из них задается обработчик (элемент из списка `mt`).

После этого мы создаем и размещаем на панели инструментов текстовую метку (с текстом **Размер шрифта**). После метки на панель добавляется спиннер. Объект спиннера `spnSize` создается на основе класса `Spinbox`. Аргументами конструктору передаются:

- ссылка на панель, в которую будет добавляться спиннер;
- аргументы `from_` и `to`, определяющие минимальное и максимальное значения, которые можно установить для спиннера;

- аргумент `font` определяет шрифт для отображения содержимого спиннера (число в поле спиннера);
- аргумент `width` (со значением `3`) задает ширину поля спиннера (в символах);
- аргумент `justify` со значением `"right"` задает способ выравнивания содержимого спиннера по правому краю;
- аргумент `textvariable` определяет «переменную» (в данном случае это `size`), значение которой соответствует значению, установленному в поле спиннера.



НА ЗАМЕТКУ

Хотя аргумент `textvariable` в названии содержит намек на текст, «переменная» `size` «целочисленная» — это объект класса `IntVar`.

Спиннер размещается на панели инструментов с помощью метода `pack()`, а затем панель инструментов размещается в окне с помощью метода `place()`.

Метод `makeFrame()` предназначен для создания вспомогательной панели с меткой, содержащей шаблонный текст. Ссылка на создаваемую при вызове метода панель записывается в поле `frame`. Метка с надписью **Пример текста** создается на основе класса `Label` и сразу добавляется на панель — метод `pack()` вызывается прямо из анонимного объекта метки.

Ссылка на метку для отображения шаблонного текста записывается в поле `lblText`. Значением аргумента `textvariable` указана ссылка на «переменную» `text`, поэтому изменение значения этой «переменной» приведет к автоматическому изменению содержимого метки. На панели метка размещается с помощью метода `pack()`, а панель в окне размещается с помощью метода `place()`.

Для создания контекстного меню использован метод `makePopupMenu()`. В общем и целом здесь много общего с созданием главного меню, но есть свои особенности. Объект контекстного меню создается на основе класса `Menu`, а ссылка на него записывается в поле `popup`. Заполняется контекстное меню последовательным вызовом методов `setMenuFont()`, `setMenuStyle()` и `setMenuColor()`, но на этот раз (в отличие от главного меню) в качестве аргумента

методам передается ссылка popup на объект контекстного меню. Команда **Выход** добавляется в контекстное меню отдельно с помощью метода add_command().

Также мы с помощью метода bind() добавляем в контекстное меню обработчик. Обработчик реагирует на событие <Button-3> (нажатие правой кнопки мыши). Обработчиком указано лямбда-выражение, в котором для аргумента evt (объект события) выполняется команда self.popup.tk_popup(evt.x_root, evt.y_root). Методом tk_popup() отображается контекстное меню, а аргументы метода evt.x_root и evt.y_root определяют координаты для отображения меню. Это координаты того места, в котором находился курсор мыши в момент наступления события. Мы эти координаты вычисляем на основе объекта события evt.

Метод apply() вызывается для того, чтобы по настройкам уже созданных элементов управления вычислить параметры шрифта и отобразить соответствующий текст в окне.

Командой clr=self.color.get() считывается цвет для шрифта. Название шрифта определяем с помощью команды nm=self.name.get(). Для определения размера шрифта использована команда sz=self.size.get().

Для применения цвета к метке задействована инструкция self.lblText.configure(fg=clr). После этого командой fnt=[nm, sz] создается список fnt с параметрами шрифта (название и размер) и начинается «уточнение» параметров шрифта и формирование шаблонного текста. Цвет (русское название) мы получаем как значение элемента словаря colors с ключом clr. Английское название для стиля шрифта (при заданном значении индекса k) дается выражением style[k][0], а русское название получаем как значение инструкции style[k][1], причем с помощью метода lower() мы преобразуем все символы в строчные (маленькие).

После того как шрифт окончательно определен, командой self.lblText.configure(font=fnt) он применяется к метке. Текст для метки задаем с помощью команды self.text.set(txt).

Метод traceVars() вызывается для того, чтобы перейти в режим отслеживания значений переменных name, color, size и переменных из списка bi.



НА ЗАМЕТКУ

Поскольку для всех «переменных» используется один и тот же метод-обработчик `apply()`, то сначала командой `mt=self.apply` в переменную `mt` записывается ссылка на этот метод, а затем переменная указывается вторым аргументом при вызове метода `trace()`.

Наконец, с помощью метода `showMainWindow()` отображается главное окно программы. Этот метод описан просто: в нем выполняется команда `self.wnd.mainloop()`.

Нам осталось проанализировать методы, которые используются для обработки событий, связанных с нажатием кнопок на панели инструментов, а также вызываемых при выборе некоторых команд меню.

В теле метода `clExit()` выполняется всего одна команда `self.wnd.destroy()`, которой закрывается окно приложения (и затем завершается выполнение программы). Метод используется для обработки нажатия на первой кнопке на панели инструментов и для обработки выбора команды **Выход** в главном и контекстном меню.

Метод `clBold()` используется как обработчик для события, связанного с нажатием кнопки применения/отмены жирного стиля. В теле метода командой `self.bi[0].set(not self.bi[0].get())` логической «переменной» `bi[0]` присваивается значение, противоположное к текущему. Поэтому если опция применения жирного стиля была установлена, то она будет отменена, и наоборот. Аналогично описан метод `clItalic()`, но подобная операция проделывается с «переменной» `bi[1]` и, соответственно, речь идет об опции применения жирного стиля.

Метод `clNormal()` вызывается при обработке нажатия кнопки отмены жирного и курсивного стиля. В теле метода командами `self.bi[0].set(False)` и `self.bi[1].set(False)` значения обеих логических «переменных» устанавливаются равными `False`.

Метод `showDialog()` предназначен для отображения модального окна справки. Мы в данном случае воспользовались стандартным методом `showinfo()` из подпакета `messagebox` из пакета `tkinter` (поэтому в заголовке программы есть инструкция `from tkinter.messagebox import *`). Аргументы метода `showinfo()` определяют заголовок модального окна и текст сообщения.

Работа с графикой

Маленькая, а с фантазией.

Из к/ф «Девчата»

Помимо использования стандартных графических компонентов, в Python существует еще и очень неплохая поддержка для использования базовых *графических примитивов* — проще говоря, при создании графического интерфейса можно еще и рисовать. Для этого используется такой графический компонент, как *область рисования*. Объект для области рисования создается на основе класса `Canvas`. У такого объекта есть целый ряд методов, позволяющих рисовать различные геометрические фигуры и задавать и изменять их настройки.

Работу с областью рисования мы проиллюстрируем на примере. Традиционно сначала рассмотрим функциональность приложения, после чего проанализируем соответствующий программный код.

При запуске приложения на выполнение отображается окно, представленное на рис. 11.22.

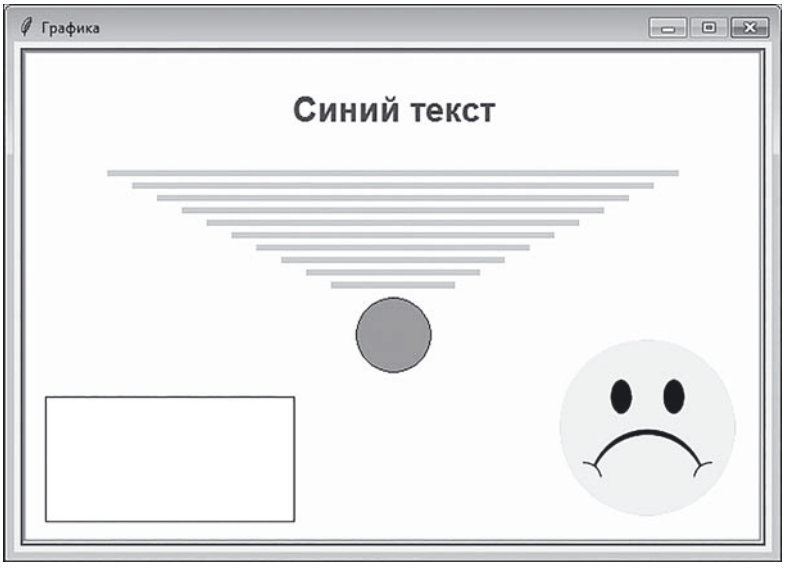


Рис. 11.22. Окно приложения сразу после запуска программы на выполнение

Окно содержит в верхней части надпись **Синий цвет**, которая отображается синим цветом. Под надписью расположена группа горизонтальных линий светло-зеленого цвета. Длина этих линий монотонно уменьшается.

НА ЗАМЕТКУ

Конечно, речь идет об отрезках. Но мы их будем называть линиями.

Под группой линий расположена окружность, закрашенная красным цветом. В левом нижнем углу рабочей области расположен прямоугольник белого цвета. В правом нижнем углу размещено «печальное» изображение. Но если навести курсор мыши на область рисования, «печальное» изображение изменится на «веселое». Более того, через точку, в которой размещен курсор, автоматически проводятся две перпендикулярные красные линии, и при движении курсора эти линии перемещаются вместе с ним. Ситуация проиллюстрирована на рис. 11.23.

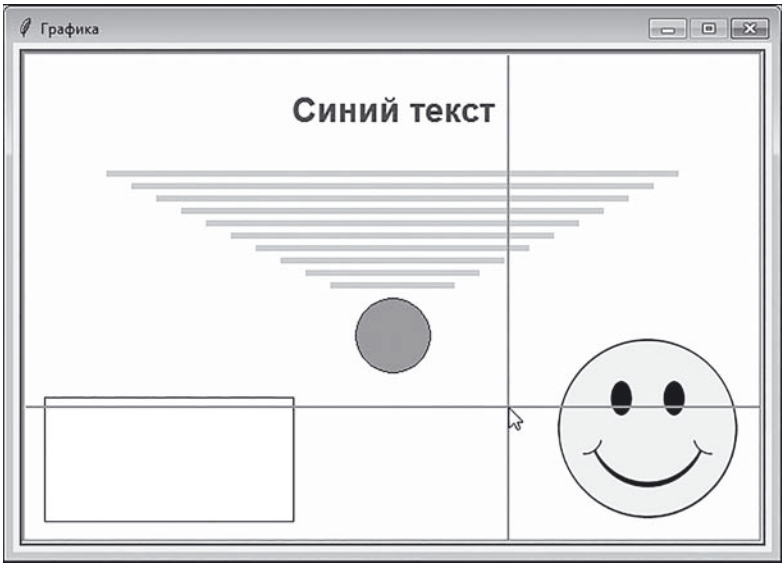


Рис. 11.23. При наведении курсора мыши на область рисования изменяется изображение в окне и отображаются две красные линии

Если курсор мыши навести на область прямоугольника, то он из белого превратится в зеленый, как показано на рис. 11.24.

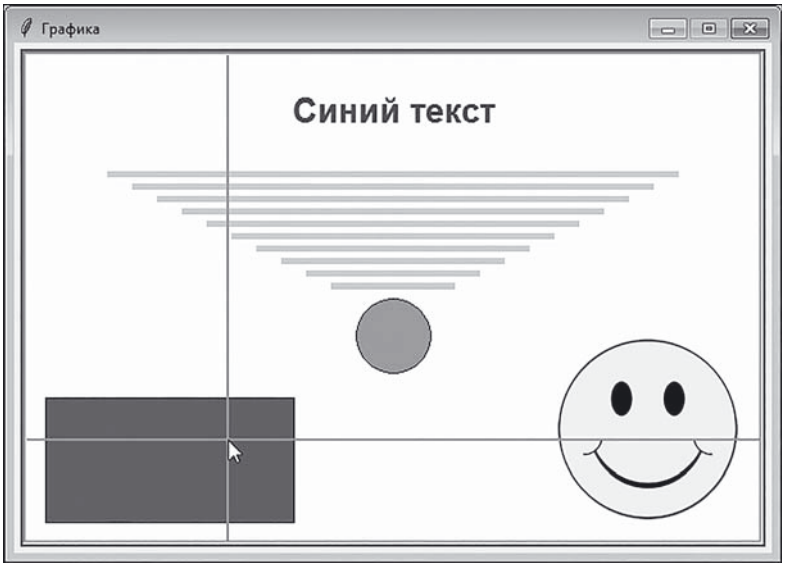


Рис. 11.24. При наведении курсора мыши на прямоугольник он закрашивается зеленым цветом

Если курсор мыши увести из области прямоугольника, прямоугольник снова станет белым. А если курсор окажется над областью окружности, то она станет белой, а прямоугольник — красным. Эта ситуация показана на рис. 11.25.

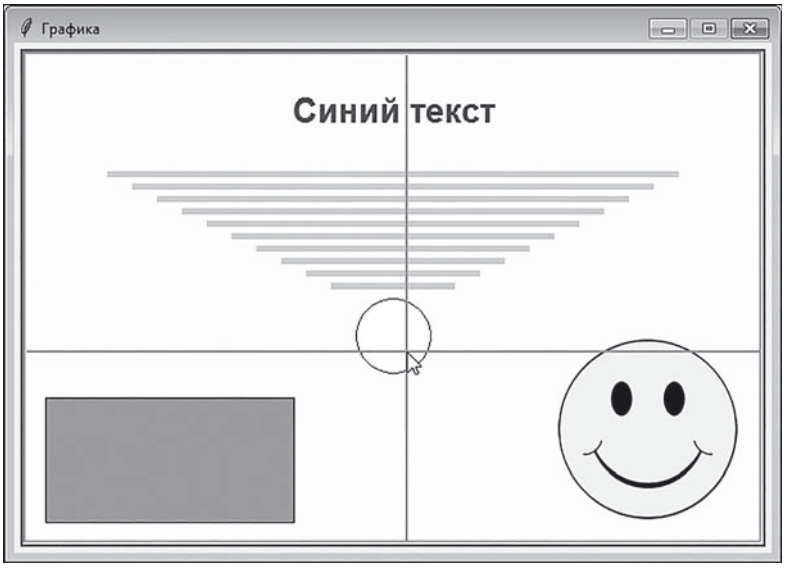


Рис. 11.25. При наведении курсора мыши на окружность она закрашивается белым цветом, а прямоугольник закрашивается красным цветом

По умолчанию фон области рисования светло-желтый. Но если щелкнуть левой кнопкой мыши в области рисования, то фон станет желтым. Чтобы фон снова стал светло-желтым, следует щелкнуть в области рисования правой кнопкой мыши.

Приложение также реагирует на нажатие кнопок со стрелками «вправо», «влево», «вверх» и «вниз». При нажатии кнопок «влево» или «вправо» в соответствующем направлении вдоль горизонтали смещается текст в верхней области окна. На рис. 11.26 показана ситуация, когда цвет фона желтый (был выполнен щелчок левой кнопкой мыши), а текст смещен влево.

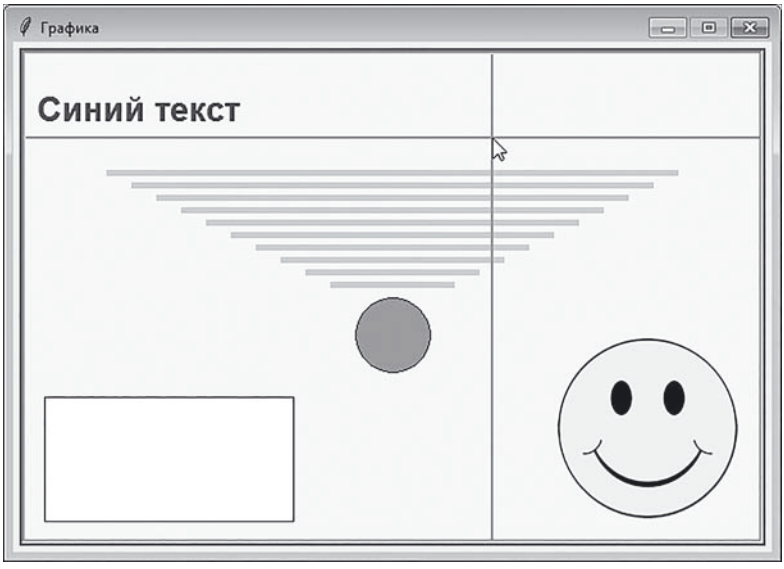


Рис. 11.26. Окно со смещенным влево текстом и желтым цветом фона

Если нажимать кнопку «вниз», то группа линий начинает смещаться вниз, а красная окружность смещается (в три раза быстрее) вверх. На рис. 11.26 показана ситуация, когда текст смещен влево, группа линий смещена вниз, окружность смещена вверх, а в качестве фона используется светло-желтый цвет (была нажата правая кнопка мыши).

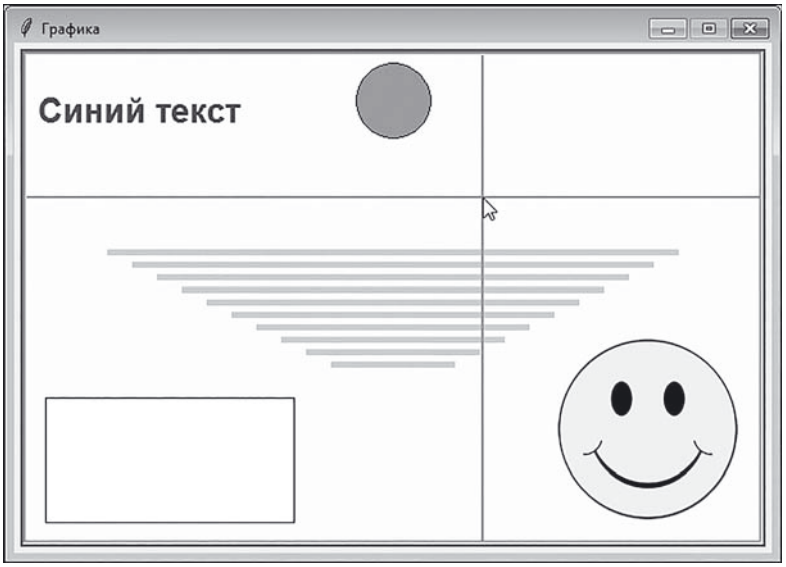


Рис. 11.27. Текст сдвинут влево, группа линий смещена вниз, окружность смещена вверх, курсор находится в области рисования

Если нажимать кнопку «вверх», то группа линий смещается вверх, а окружность смещается вниз. На рис. 11.28 проиллюстрирована ситуация, когда текст смещен вправо, группа линий смещена вверх, окружность смещена вниз, а курсор покинул область рисования.

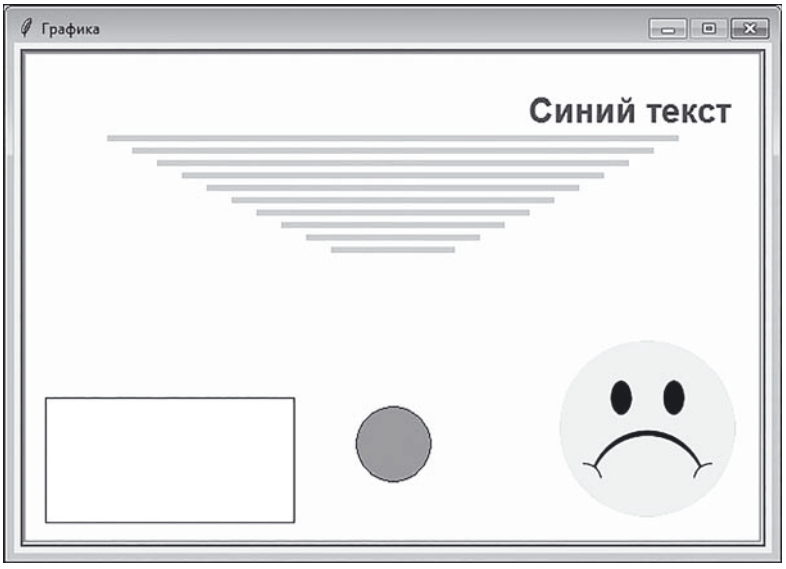


Рис. 11.28. Текст сдвинут вправо, группа линий смещена вверх, окружность смещена вниз, курсор находится вне области рисования

Теперь рассмотрим и проанализируем программный код, представленный в листинге 11.7.

Листинг 11.7. Работа с графикой

```
from tkinter import *
from math import *
# Функция для обработки события, связанного
# с наведением курсора на область рисования:
def msEnter(evt):
    # Смена изображения:
    cnv.itemconfig(Pct, image=img_2)
# Функция для обработки события, связанного
# с тем, что курсор покидает область рисования:
def msLeave(evt):
    # Смена изображения:
    cnv.itemconfig(Pct, image=img_1)
    # Удаление "старых" линий возле курсора:
    cnv.delete("ln")
# Функция для обработки события, связанного
# с перемещением курсора в области рисования:
def msMotion(evt):
    # Координаты курсора в области рисования:
    x=evt.x
    y=evt.y
    # Удаление "старых" линий:
    cnv.delete("ln")
    # Отображение линий:
    cnv.create_line(x,5, x, H-5, fill=clr_C1, width=2, tag="ln")
    cnv.create_line(5, y, W-5, y, fill=clr_C1, width=2, tag="ln")
    # Координаты прямоугольника:
    Rxy=cnv.coords(Rtn)
    # Координаты окружности:
    Cxy=cnv.coords(Crl)
```

```
    # Координаты центра окружности:
    x0=(Cxy[2]+Cxy[0])/2
    y0=(Cxy[3]+Cxy[1])/2
    # Расстояние от курсора до центра окружности:
    r=sqrt((x-x0)**2+(y-y0)**2)
    # Если курсор находится внутри окружности:
    if r<R:
        # Белый цвет заливки для окружности:
        cnv.itemconfig(Crl, fill=clr_C2)
        # Красный цвет заливки для прямоугольника:
        cnv.itemconfig(Rtn, fill=clr_C1)
        # Завершение выполнения функции:
        return
    # Если курсор находится вне окружности:
    else:
        # Красный цвет заливки для окружности:
        cnv.itemconfig(Crl, fill=clr_C1)
    # Если курсор находится внутри прямоугольника:
    if x>Rxy[0] and x<Rxy[2] and y>Rxy[1] and y<Rxy[3]:
        # Зеленый цвет заливки для прямоугольника:
        cnv.itemconfig(Rtn, fill=clr_R2)
    # Если курсор находится вне прямоугольника:
    else:
        # Белый цвет заливки для прямоугольника:
        cnv.itemconfig(Rtn, fill=clr_R1)
# Функция для обработки события, связанного
# с нажатием кнопки "вверх":
def msUp(evt):
    # Группа линий смещается на один пиксель вверх:
    cnv.move("Lns",0,-1)
    # Окружность смещается на три пикселя вниз:
    cnv.move(Crl,0,3)
```

```
# Функция для обработки события, связанного
# с нажатием кнопки "вниз":
def msDown(evt):
    # Группа линий смещается на один пиксель вверх:
    cnv.move("Lns",0,1)
    # Окружность смещается на три пикселя вверх:
    cnv.move(Crl,0,-3)
# Функция для обработки события, связанного
# с нажатием кнопки "влево":
def msLeft(evt):
    # Текст смещается на одну позицию влево:
    cnv.move(Txt,-1,0)
# Функция для обработки события, связанного
# с нажатием кнопки "вправо":
def msRight(evt):
    # Текст смещается на одну позицию вправо:
    cnv.move(Txt,1,0)
# Ширина и высота области рисования:
W=600
H=400
# Ширина и высота прямоугольника:
w=200
h=100
# Количество линий:
N=10
# Декремент для длины линий:
d=20
# Радиус окружности:
R=30
# Шрифт:
fnt=("Arial",20,"bold")
# Текст:
```

```
txt="Синий текст"
# Цвет для линий:
clr="lightgreen"
# Цвет для фона области рисования:
clr_1="lightyellow"
clr_2="yellow"
# Цвет для закраски окружности:
clr_C1="red"
clr_C2="white"
# Цвет для закраски прямоугольника:
clr_R1="white"
clr_R2="green"
# Создание объекта окна:
wnd=Tk()
# Размеры и положение окна:
wnd.geometry(str(W+10)+"x"+str(H+10)+"+400+300")
# Заголовок для окна:
wnd.title("Трафика")
# Окно постоянных размеров:
wnd.resizable(False, False)
# Создание объекта для области рисования:
cnv=Canvas(wnd, bg=clr_1, bd=3, relief=GROOVE)
# Размещение области рисования в окне:
cnv.place(x=5, y=5, width=W, height=H)
# Передача фокуса области рисования:
cnv.focus_set()
# Создание текстового элемента:
Txt=cnv.create_text(W/2,50, text=txt, font=fnt, fill="blue")
# Создание группы горизонтальных линий:
for k in range(N):
    # Координаты линий:
    x1=70+k*d
    y1=H/4+10*k
    x2=W-70-d*k
```

```
    y2=H/4+10*k
    cnv.create_line(x1, y1, x2, y2, fill=clr, width=5, tag="Lns")
# Координаты для создания окружности:
x1=W/2-R
y1=H/2-R+30
x2=W/2+R
y2=H/2+R+30
# Создание окружности:
Crl=cnv.create_oval(x1, y1, x2, y2, fill=clr_C1)
# Координаты для создания прямоугольника:
x1=20
y1=H-h-20
x2=x1+w
y2=y1+h
# Создание прямоугольника:
Rtn=cnv.create_rectangle(x1, y1, x2, y2, fill=clr_R1)
# Создание объектов изображений:
img_1=PhotoImage(file="D:\\Pictures\\sad.png")
img_2=PhotoImage(file="D:\\Pictures\\smile.png")
# Создание объекта изображения:
Pct=cnv.create_image(W-20, H-20, anchor=SE, image=img_1)
# Регистрация обработчиков:
cnv.bind("<Left>", msLeft)
cnv.bind("<Right>", msRight)
cnv.bind("<Up>", msUp)
cnv.bind("<Down>", msDown)
cnv.bind("<Enter>", msEnter)
cnv.bind("<Leave>", msLeave)
cnv.bind("<Motion>", msMotion)
cnv.bind("<Button-1>", lambda evt: cnv.config(bg=clr_2))
cnv.bind("<Button-3>", lambda evt: cnv.config(bg=clr_1))
# Отображение главного окна:
wnd.mainloop()
```

Удобства ради в программе создается несколько переменных (ширина и высота области рисования W и H, ширина и высота прямоугольника w и h, количество линий в группе линий N, декремент для длины линий d, радиус окружности R, шрифт fnt, текст для отображения в области рисования txt, а также переменные clr, clr_1, clr_2, clr_C1, clr_C2, clr_R1 и clr_R2, определяющие цвет).

После создания объекта окна wnd задаются его размеры и положение командой wnd.geometry(str(W+10)+"x"+str(H+10)+"+400+300"). В текстовом аргументе метода geometry() два последних числа (400 и 300), указанные через символ "+", определяют положение окна в области экрана: это координаты левого верхнего угла окна по отношению к левому верхнему углу экрана.

Объект cnv для области рисования создается командой cnv=Canvas(wnd, bg=clr_1, bd=3, relief=GROOVE). Аргументы конструктора определяют принадлежность области рисования окну wnd, цвет фона (аргумент bg), толщину рамки вокруг области (аргумент bd) и тип рамки (аргумент relief). Для размещения области рисования в окне использован метод place(). Командой cnv.focus_set() области рисования передается фокус.

Для создания текстового элемента (это фактически «нарисованная» надпись) Txt использован метод create_text(). Первые два аргумента метода определяют место размещения текстовой надписи. Собственно текст задается аргументом text, шрифт определяется аргументом font, а цвет текста задается с помощью аргумента fill.

Для создания группы горизонтальных линий использован оператор цикла. Каждая отдельная линия создается с помощью метода create_line(). Первые четыре аргумента определяют координаты начальной и конечной точек, которые соединяются линией. С помощью аргумента fill определяется цвет линий, а их толщина определяется значением аргумента width. Также мы используем аргумент tag, который задает дескриптор (или тег), по которому затем можно будет получать доступ к линиям. Поскольку для всех линий указан один и тот же дескриптор "Lns", то с помощью этого дескриптора мы впоследствии сможем управлять группой линий как одним целым.

Для создания окружности Crl использован метод create_oval(). Вообще метод предназначен для рисования овалов. В качестве аргументов методу передаются координаты левого верхнего и правого нижнего

углов воображаемого прямоугольника, в который вписывается овал. Если прямоугольник является квадратом, то получим окружность. Мы, в данном случае, при вызове метода `create_oval()`, кроме координат указываем еще и значение для аргумента `fill=clr_C1`. Это цвет, которым закрашивается окружность.

Аналогично создается прямоугольник `Rtn`, но на этот раз использован метод `create_rectangle()`.

Также в программе есть пример отображения в области рисования изображения из файла. Для этого мы на основе класса `PhotoImage` и файлов с изображениями создаем два объекта `img_1` и `img_2`. Объект изображения `Pct` создается с помощью метода `create_image()`. Первые два аргумента определяют координаты точки в области рисования, в которой размещается изображение. Аргумент `anchor` определяет способ «привязки» изображения к точке, заданной координатами. Значение `SE` означает, что в соответствующей точке будет размещен правый нижний угол области изображения.



НА ЗАМЕТКУ

Значение `SE` получается от англоязычных названий сторон света — в данном случае это South East (юго-восток, что соответствует правому нижнему углу). Значение `NW`, например, означает North West (северо-запад) — левый верхний угол.

Значением аргумента `image` указывается ссылка на отображаемую картинку.

С помощью метода `bind()` для разных событий, связанных с областью рисования `cnv`, регистрируются обработчики: в основном это функции, но есть и два лямбда-выражения. Вся «динамика» связана с этими обработчиками.



НА ЗАМЕТКУ

У всех функций один аргумент (называется `evt`), который отождествляется с объектом обрабатываемого события.

Функция `msEnter()` используется для обработки события `<Enter>`, связанного с наведением курсора на область рисования. В теле функции командой `cnv.itemconfig(Pct, image=img_2)` выполняется смена изображения, которое рисуется в правом нижнем углу области рисования.

ПОДРОБНОСТИ

В программе используется метод `itemconfig()`. Метод вызывается из объекта `cnv`, а первым аргументом методу передается ссылка на объект, для которого выполняется настройка параметров. Далее указываются значения опций, которые применяются для объекта, указанного в качестве первого аргумента.

Функция `msUp()` используется как обработчик события `<Up>`, связанного с нажатием кнопки «вверх». В теле функции командой `cnv.move("Lns",0,-1)` группа линий смещается на один пиксель вверх. Первым аргументом метода `move()` указывается дескриптор `"Lns"`, общий для всех линий в группе, поэтому операция (перемещения) выполняется для каждой линии в группе. Значения `0` и `-1` означают, что изображение смещается по горизонтали на `0` пикселей и по вертикали на `-1` пиксель. Начало координатной системы области рисования находится в левом верхнем углу этой области. Координатные оси направлены слева направо (горизонтальная) и сверху вниз (вертикальная). Поэтому значение `-1` для смещения по вертикали означает, что изображение смещается на `1` пиксель вверх (в противоположном направлении к направлению вертикальной координатной оси). А командой `cnv.move(Crl,0,3)` окружность смещается по горизонтали на `0` пикселей и на `3` пикселя (вниз) вдоль вертикали.

Функция `msDown()` нужна для обработки события `<Down>`, связанного с нажатием кнопки «вниз». В теле функции выполняются команды `cnv.move("Lns",0,1)` и `cnv.move(Crl,0,-3)`, в результате чего группа линий смещается на `1` пиксель вниз, а окружность смещается на `3` пикселя вверх.

Для обработки события `<Left>` (нажатие кнопки «влево») используется функция `msLeft()`. В ней командой `cnv.move(Txt,-1,0)` текст смещается на `1` пиксель влево. В функции `msRight()` (используется для обработки события `<Right>`, связанного с нажатием кнопки «вправо») командой `cnv.move(Txt,1,0)` текст смещается на один пиксель вправо.

Самая «объемная» функция `msMotion()` регистрируется как обработчик события `<Motion>`, связанного с перемещением курсора в области рисования. Сначала командами `x=evt.x` и `y=evt.y` на основе объекта события `evt` вычисляются координаты курсора (на момент возникновения события) в области рисования. Далее командой `cnv.`

delete("ln") удаляются «старые» красные линии, которые были нарисованы при предыдущем вызове функции. А затем командами cnv.create_line(x,5, x, H-5, fill=clr_C1, width=2, tag="ln") и cnv.create_line(5, y, W-5, y, fill=clr_C1, width=2, tag="ln") рисуются две новые линии, которые проходят через точку, в которой находится курсор.

ПОДРОБНОСТИ

При вызове функции рисуются две линии. Функция вызывается каждый раз, когда курсор меняет свою позицию в области рисования. При этом старые линии автоматически не удаляются. Работу по удалению этих уже не нужных линий следует возложить на функцию. Обе линии имеют один и тот же дескриптор "ln". Это позволяет удалить сразу две линии, указав дескриптор в качестве аргумента метода delete().

На следующем этапе мы решаем еще несколько вспомогательных задач. Так, при выполнении команды Rxy=cnv.coords(Rtn) в переменную Rxy записывается ссылка на список, содержащий координаты прямоугольника Rtn. Это координаты левого верхнего и правого нижнего углов прямоугольника. Аналогичные последствия выполнения команды Cxy=cnv.coords(Crl): в переменную Cxy записывается ссылка на список с координатами воображаемого квадрата, в который вписана окружность Crl. Затем с помощью команд x0=(Cxy[2]+Cxy[0])/2 и y0=(Cxy[3]+Cxy[1])/2 мы вычисляем координаты точки, являющиеся центром окружности (эта же точка — центр квадрата, в который вписана окружность). Расстояние от точки, в которой находится курсор, до центра окружности вычисляется командой r=sqrt((x-x0)**2+(y-y0)**2).

ПОДРОБНОСТИ

Для вычисления квадратного корня использована математическая функция sqrt() из модуля math. Для возможности использования функции в заголовке программы размещена инструкция from math import *.

Затем в дело вступает условный оператор, в котором проверяется условие r<R (курсор находится в области окружности). Если так, то командой cnv.itemconfig(Crl, fill=clr_C2) для окружности приме-

няется белый цвет заливки, а командой `cnv.itemconfig(Rtn, fill=clr_C1)` красный цвет заливки применяется для прямоугольника, после чего с помощью инструкции `return` завершается выполнение функции (поскольку нет необходимости проверять, находится ли курсор над областью прямоугольника).

Если же курсор находится вне окружности, то командой `cnv.itemconfig(Crl, fill=clr_C1)` для окружности применяется красный цвет заливки. После этого с неизбежностью выполняется еще один условный оператор. В нем использовано условие `x>Rxy[0] and x<Rxy[2] and y>Rxy[1] and y<Rxy[3]`. Его истинность означает, что координаты курсора попадают внутрь области прямоугольника. В этом случае с помощью команды `cnv.itemconfig(Rtn, fill=clr_R2)` для прямоугольника применяется зеленый цвет заливки. А если курсор не находится в области прямоугольника, то с помощью команды `cnv.itemconfig(Rtn, fill=clr_R1)` для прямоугольника применяется белый цвет заливки.

Для обработки события `<Leave>` (курсор покидает область рисования) мы используем функцию `msLeave()`. В теле функции всего две команды. Командой `cnv.itemconfig(Pct, image=img_1)` выполняется смена изображения в правом нижнем углу области рисования. Командой `cnv.delete("ln")` удаляются линии, объединенные дескриптором `"ln"` (то есть линии, которые «остались» от курсора, который покинул область рисования).



НА ЗАМЕТКУ

Дело в том, что, когда курсор покидает область рисования, на ней может остаться (и скорее всего, останется) как минимум одна из красных линий, которые «сопровождали» курсор. Поэтому эта линия (или линии) удаляются.

События `<Button-1>` возникает, когда пользователь нажимает левую кнопку мыши в области рисования (в данном случае). Обработчиком события указано лямбда-выражение. В нем выполняется команда `cnv.config(bg=clr_2)`, которой задается цвет фона для области рисования. Аналогично обрабатывается событий `<Button-3>` (нажатие правой кнопки мыши в области рисования). Разница лишь в том, какой цвет применяется для фона области рисования.

Резюме

— Интурист хорошо говорит.
— А что он говорит, конкретно что?

Из к/ф «Иван Васильевич меняет профессию»

- Создание программ с графическим интерфейсом подразумевает создание объектов для графических компонентов, их компоновку в контейнере (окне или панели), а также обработку событий.
- Объект главного окна создается на основе класса `Tk`. Также часто используются панели (объекты класса `Frame`), кнопки (объекты класса `Button`), метки (объекты класса `Label`), поля ввода (объекты класса `Entry`), списки (объекты класса `Listbox`), опции (объекты класса `Checkbutton`), переключатели (объекты класса `Radiobutton`) и многие другие. Помимо этого для приложений можно создавать меню (используется класс `Menu`), в том числе и контекстное.
- Для настройки параметров компонентов используются специальные методы, а также задаются значения соответствующих аргументов. Например, для размещения компонентов в контейнерах используются методы `place()`, `pack()` и `grid()`. Для настройки параметров компонентов используется метод `configure()`.
- Для связывания методов и функций, предназначенных для обработки событий, с компонентами используют метод `bind()` или задают значение аргумента `command`. Также можно использовать объекты специальных классов `StringVar`, `BooleanVar`, `IntVar` и `DoubleVar`.
- Для работы с графикой на основе класса `Canvas` создается объект для области рисования. Этот объект содержит методы для создания различных геометрических фигур, таких, например, как линии, прямоугольники, окружности и ряд других. Для таких графических изображений можно выполнять обработку событий.